Linux Systems and Performance

PART 3: Memory Management

Week 8: Virtual Memory to Physical Memory (Kernel View)

Sungyong Park, Ph.D.

Data-Intensive AI Computing and Systems (DISCOS) Laboratory

Department of Computer Science and Engineering, Sogang University

URL: https://discos.sogang.ac.kr

E-mail: parksy@sogang.ac.kr

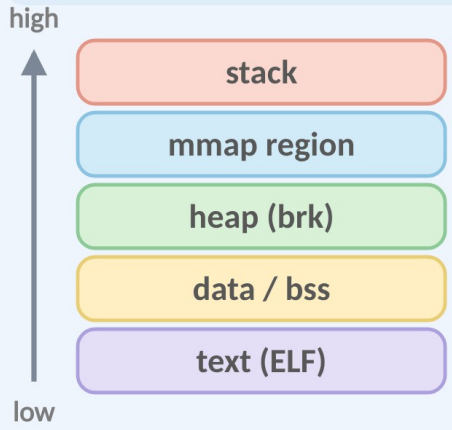


Roadmap of Part 3

This week

7 Virtual memory (application view)

How does a process see and use its memory?



- Process address space and ELF mappings
- VMAs, brk() vs mmap()
- malloc() and the glibc allocator

8 Address translation (kernel view)

How does a virtual address become a physical page?

Virtual address
from the CPU

↓

Page table walk
TLB lookup first

↓

Physical page
DRAM

- Page tables and the TLB
- Page faults (minor and major)
- Buddy and slab allocators

9 Memory pressure and page reclaim

How does Linux recover pages under pressure?

Memory pressure
free pages run low

↓

Reclaim pages
kswapd / direct

↓

Free pages restored
allocation succeeds

- Watermarks, kswapd vs direct reclaim
- LRU, swap and zram
- Observing pressure (PSI, cgroup)

10 Memory shortage, recovery, and OOM

What happens when reclaim cannot keep up?

Reclaim cannot keep up
severe pressure

↓

OOM killer / LMK
pick a victim

↓

Memory recovered
system keeps running

- OOM killer: who dies, and why
- LMK, PSI and user-space killers
- Finding the cause: leaks, fragmentation

PART 3: Memory Management

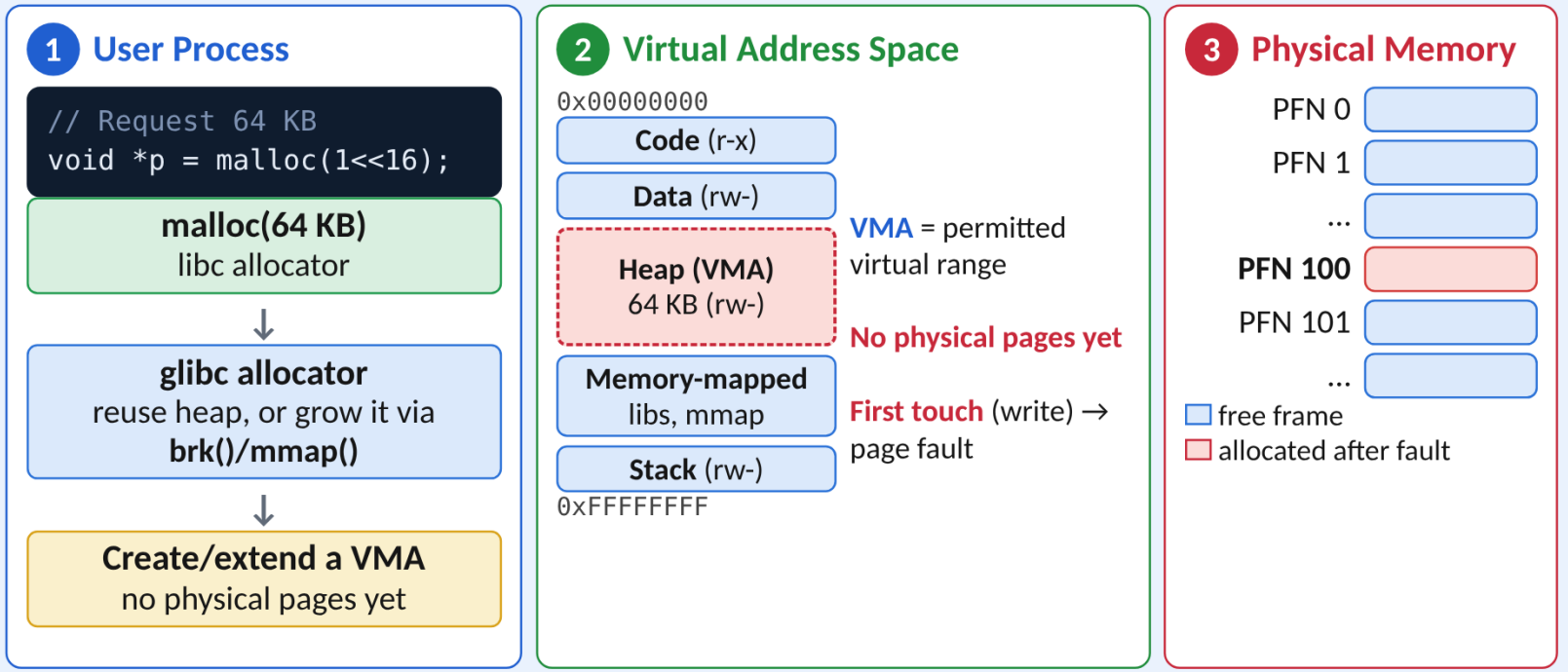
- **Week 8**

- **Address Translation, Paging, and TLB**
  - How does a virtual address translate into a physical memory access?
- **Page Fault and Physical Page Allocation**
  - How does Linux obtain and map a physical page when a page fault occurs?
- **Buddy Allocator, SLUB, and Kernel Object Allocation**
  - How does Linux efficiently allocate and manage physical pages and kernel objects?

Recap: From VMA to Physical Memory

Key Points

- **Virtual first**
  A call to **malloc()/new** first allocates a region in the virtual address space (a **VMA**).
- **Not backed yet**
  Memory is **not** backed by physical pages immediately; pages are allocated **on demand** (first touch).
- **Kernel maps pages**
  Uses **page tables**; pages come from the **buddy allocator**.
- **Page fault**
  On access, a **page fault** allocates a free page, updates the page table, and resumes execution.



Recap: Week 7

malloc()/new → libc → **brk()/mmap()** create or extend a **VMA**: a permitted range, not yet backed.

1 Page tables

The kernel maps each **VPN** to a **PFN** through **PTEs**.

2 Page faults

First access to an unmapped page traps into the kernel, which allocates a frame, **updates the page table**, and resumes.

3 Allocators

Frames come from the **buddy allocator**; kernel objects from **SLUB**.

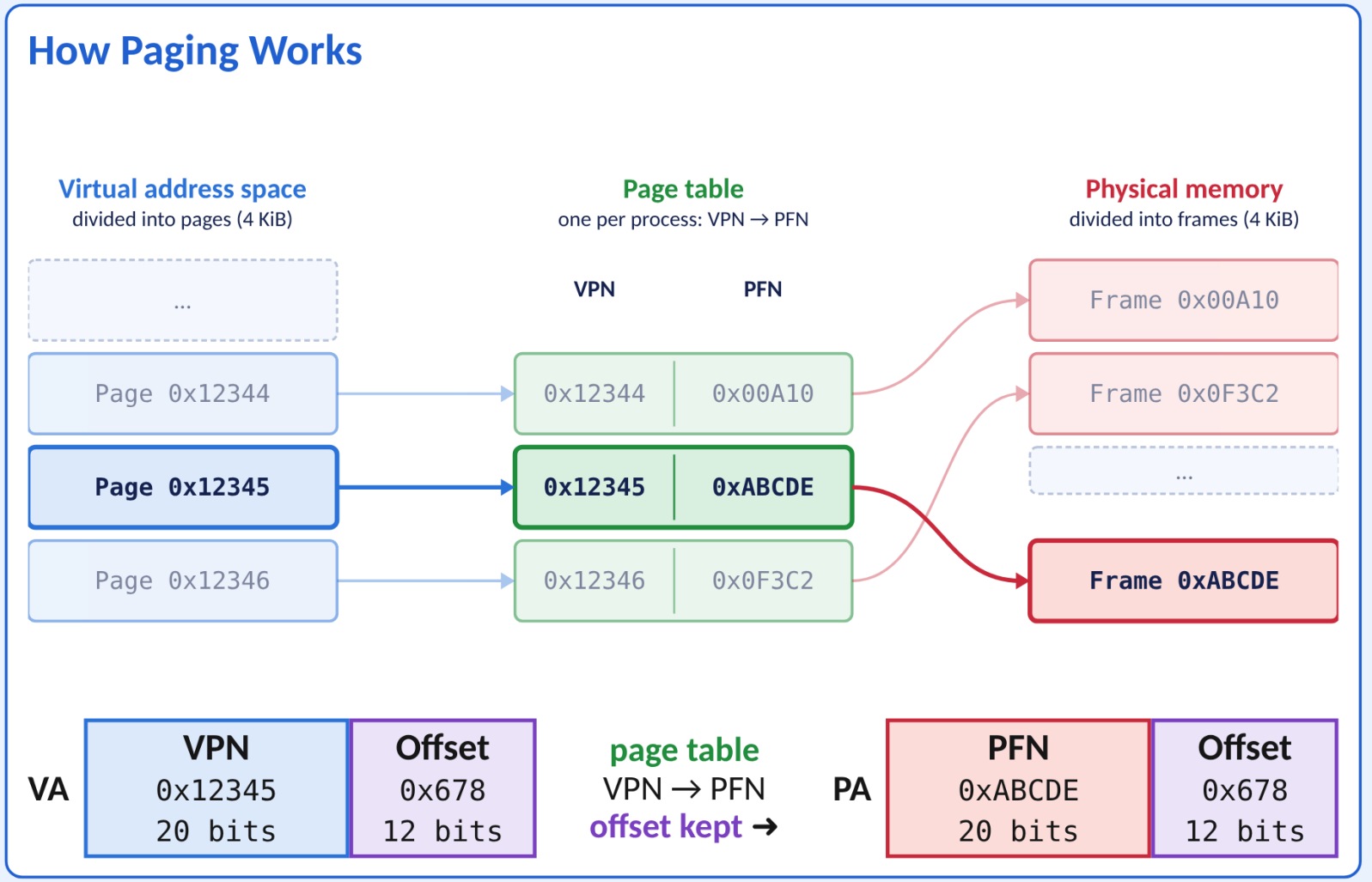
Key Takeaway

malloc()/new creates a VMA first; physical pages are typically allocated on demand at first touch via page faults.

How Paging Works

Key Points

- **Pages and frames**
  Virtual space is split into **pages**, physical memory into **frames**, both the **same size** (4 KiB).
- **Address = number + offset**
  A VA is a **page number (VPN)** plus an **offset** inside the page.
- **Page table**
  Per process, it maps each **VPN** to a **PFN**.
- **Offset kept**
  PA = PFN + same offset: only the number changes.
- **Any frame**
  Consecutive pages can use **non-contiguous** frames.



Key Takeaway

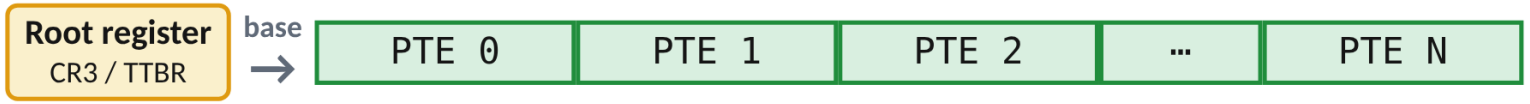
Paging splits both spaces into equal-size units; translation swaps the page number for a frame number and keeps the offset.

Page Table Structure: The Flat Table and Its Problems

Key Points

- **Flat Page Table**
  The kernel stores it in memory as **one flat array** with one entry per page.
- **Problem 1: Size**
  With a **48-bit** address and **4 KiB** pages, it needs **512 GiB**.
- **Problem 2: Contiguity**
  The array must be **one contiguous block**.
- **Problem 3: Sparseness**
  Most entries are **unused**, yet all are reserved.
- **Next Question**
  Can **bigger pages** fix these problems?

Flat Page Table: One Array of PTEs



One array, indexed by VPN: **PTE address = base + VPN × PTE size**. Stored in memory, **one per process**.

1 Size

Assume 4 KiB pages: #PTEs = 2^{VA-12}

32-bit	2^{20} PTEs × 4 B	**4 MiB**
48-bit	2^{36} PTEs × 8 B	**512 GiB**
64-bit	2^{52} PTEs × 8 B	**32 PiB**

Per process. 512 GiB is **32×** a 16 GiB RAM.

2 Contiguity

The array must be **one contiguous block** of physical memory: PTE address is **base + offset**.

free memory is **scattered**



512 GiB in one piece is **hard to find** (fragmentation).

3 Sparseness

A process maps **little** of its address space; **most PTEs are empty**, yet the flat table reserves them all.



red = in use (code, heap, stack, ...), gray = unused

48-bit VA, 4 KiB pages: 8 MiB mapped = **2,048 of 2^{36}** PTEs in use (**0.000003%**).

Key Takeaway

A flat page table is huge, must be contiguous, and is mostly empty: can bigger pages fix them?

What If We Increase the Page Size?

Key Points

- **Bigger Pages**
  Fewer entries are needed, so the flat table gets **smaller**.
- **Contiguity**
  The table is still **one contiguous array**.
- **Sparseness**
  Most entries are still **empty** but reserved.
- **New Costs**
  Memory is wasted by **internal fragmentation**.
- **Next Step**
  Change the **structure**: use a multi-level table.

Bigger Pages Make the Flat Table Smaller

Page size	#PTEs (48-bit VA)	Flat table size	Frame to find
4 KiB	$2^{36} \times 8$ B	**512 GiB**	4 KiB
2 MiB	$2^{27} \times 8$ B	**1 GiB**	2 MiB contiguous
1 GiB	$2^{18} \times 8$ B	**2 MiB**	1 GiB contiguous

#PTEs = $2^{48 - \log_2(\text{page size})}$. **Bigger pages → fewer PTEs → smaller table.**

Contiguity: Still There

The table is still **one contiguous array**: **1 GiB** at 2 MiB pages.

And now every **page needs a contiguous 2 MiB / 1 GiB frame**: harder to find.

Sparseness: Still There

Most PTEs are **still empty**, and still reserved.



48-bit VA, 2 MiB pages: 8 MiB mapped = **4 of 2^{27}** PTEs in use (**0.000003%**).

New Costs

Internal fragmentation: a 4 KiB mapping still takes a **whole 2 MiB** page.

Coarse granularity: one protection setting, one swap/COW unit per page.

Key Takeaway

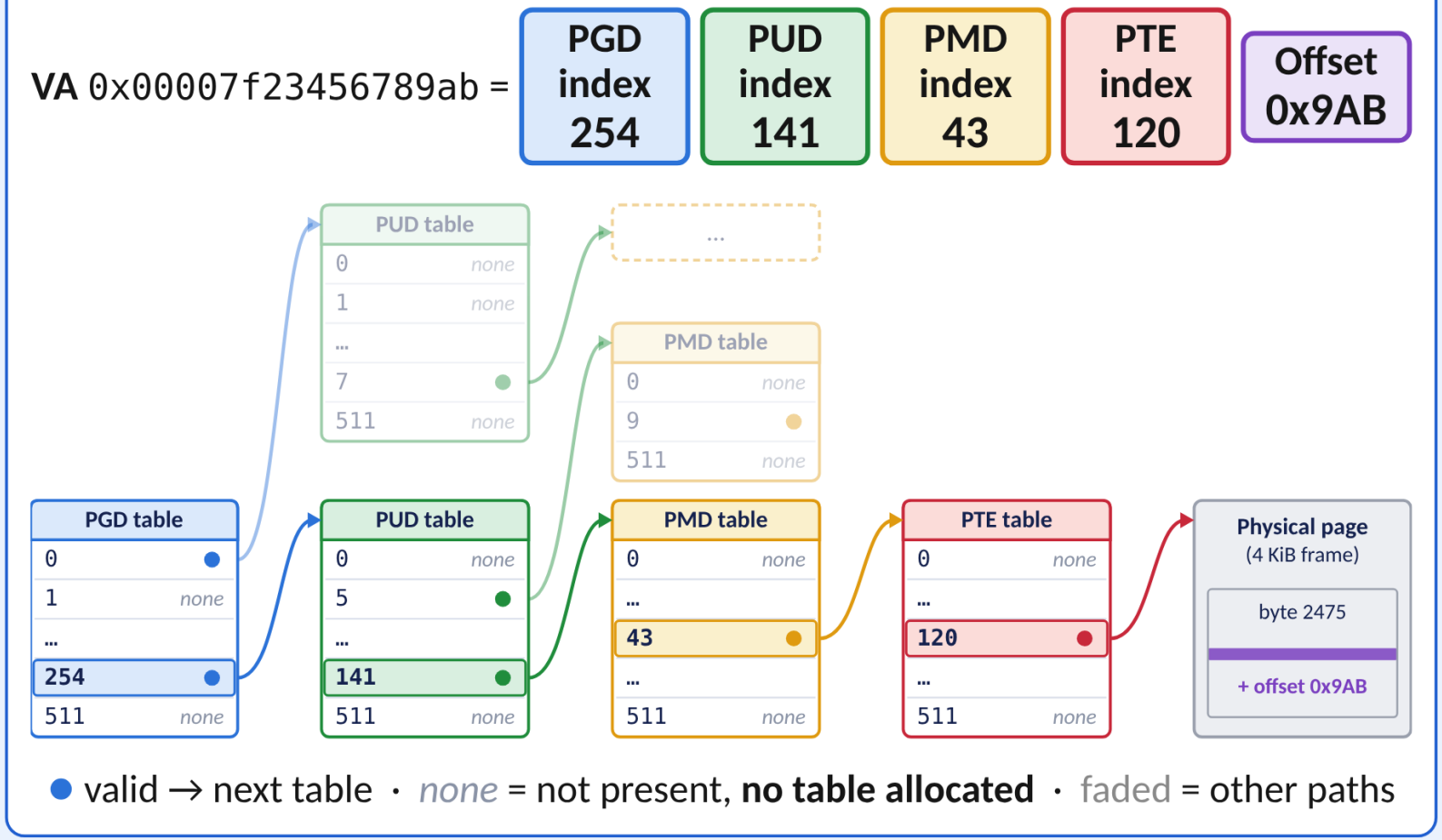
Bigger pages only shrink the flat table; contiguity and sparseness remain, so we must change the structure: multi-level.

How a Multi-Level Page Table Works

Key Points

- **Tree of Tables**
  Each level is a **page-sized table** of 512 entries that points to the next level.
- **Valid or Empty**
  A **valid entry** leads down; an **empty entry** means that subtree does not exist.
- **Walking the Tree**
  One **index per level** picks an entry; the last one gives the physical page, and the offset picks the byte.
- **Why It Helps**
  Tables are **non-contiguous** and exist **only where memory is mapped**.

Multi-Level Page Table: A Tree of 512-Entry Tables



Key Takeaway

Multi-level tables form a tree: each index picks one entry, and a none entry means a whole subtree needs no memory.

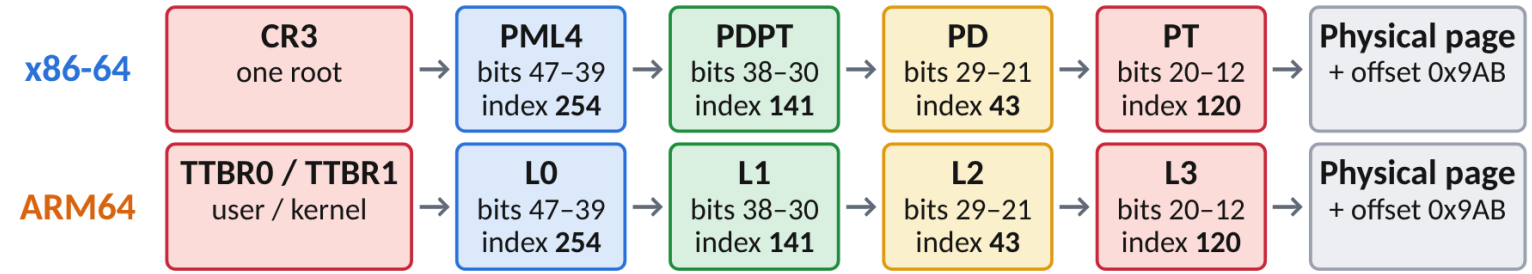
x86-64 vs. ARM64 Address Translation

Key Points

- **Same Structure**
  Both use a **4-level walk**: 9+9+9+9+12 bits.
- **Different Names**
  x86-64: **PML4 to PT**; ARM64: **L0 to L3**; Linux: **PGD to PTE**.
- **Root Register**
  x86-64 uses **CR3**; ARM64 uses **TTBR0 and TTBR1**.
- **32-bit Process**
  The same tables work; the **top levels use few entries**.
- **Page Fault**
  A missing or forbidden entry raises a **page fault**.

1 Page Table Walk: x86-64 vs. ARM64 (48-bit VA, 4 KiB pages)

Same VA **`0x00007f23456789ab`**: **9+9+9+9+12** bits, 512 entries per table. Linux: **PGD→PUD→PMD→PTE**



Entry not present or not permitted → **page fault** on both.

2 What Differs

	x86-64	ARM64
Root register	CR3 (one root per address space)	TTBR0_EL1 (user), TTBR1_EL1 (kernel)
User / kernel	Kernel in the upper half of the same tables	Separate kernel table; VA top bits pick TTBR
Page sizes	4 KiB (+ 2 MiB, 1 GiB huge)	4 / 16 / 64 KiB granule
Fault info	CR2 + error code	ESR_EL1 + FAR_EL1
32-bit process (VA < 4 GiB)	PML4: entry 0 only; PDPT: entries 0–3	L0: entry 0 only; L1: entries 0–3

Key Takeaway

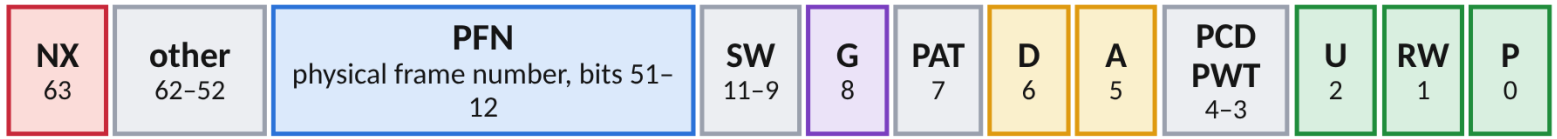
x86-64 and ARM64 share the same 4-level 9+9+9+9+12 walk; they differ in root registers, kernel mapping, and fault reporting.

What is Inside a PTE?

Key Points

- **Entry Layout**
  A page table entry is **64 bits**: a **frame number** plus **flag bits**.
- **Present Bit**
  P = 0 means not mapped yet, so an access causes a **page fault**.
- **Protection Bits**
  RW, U, and NX control what is allowed.
- **Accessed and Dirty**
  The **hardware sets** them; the OS reads them.
- **Page Size and Global**
  PS maps huge pages; **G** keeps kernel TLB entries.

1 x86-64 PTE Layout (4 KiB page)



A **64-bit** entry: **PFN** plus **flag bits**. Colored cells are used in the next slides.

2 The Bits We Will Use

Flag	Bit	Meaning	Used later for
P	0	Present: 1 = valid, 0 = not present	Page fault: demand paging, swap
RW	1	Writable (0 = read-only)	Protection, copy-on-write
U	2	User-accessible (0 = supervisor only)	Kernel / user isolation
A	5	Accessed: set by hardware on access	Reclaim: which pages are cold
D	6	Dirty: set by hardware on write	Writeback of modified pages
NX	63	No-execute (1 = not executable)	Protection (no code in data pages)
G	8	Global: not flushed on address-space switch	Kernel TLB entries
PS*	7	*PMD / PUD entry maps a huge page	Huge pages (2 MiB / 1 GiB)

ARM64: same ideas, different layout (AP, UXN/PXN, AF, nG).

Key Takeaway

A PTE holds the PFN and flag bits; P, RW/U/NX, A/D, G, and PS are the ones we use in the following slides.

PTE Flags in Action: Protection and More

Key Points

- **Every Access**
  The Memory Management Unit checks the flags on **every access**.
- **Protection Violation**
  A wrong **RW, U, or NX** access raises a **page fault**.
- **Not Present**
  P = 0 also raises a page fault; the OS decides what to do.
- **Accessed and Dirty**
  The hardware sets them; the OS uses them for **reclaim** and **writeback**.
- **Hardware Checks**
  All checks run in **hardware**, not software.

1 Protection: Hardware Checks RW, U, NX

`STORE *p = 0x11`	`User-mode LOAD`	`Instruction FETCH`
↓	↓	↓
RW = 0	U = 0	NX = 1
↓	↓	↓
Denied → #PF	Denied → #PF	Denied → #PF
A valid mapping does not imply write permission.	Supervisor-only pages are blocked in user mode.	Data pages cannot be executed.

2 Other Uses of PTE Flags

P = 0: not present
Access raises **#PF**; the OS **allocates** the page, **reads** it from swap or file, or sends **SIGSEGV**.

A and D: hardware sets
A on any access, **D** on write. The OS reads them to find **cold pages** and pages needing **writeback**.

PS and G: later
PS = 1 ends the walk early (huge page). **G = 1** keeps kernel TLB entries across switches.

Key Takeaway

The MMU checks PTE flags on every access: protection violations and not-present pages raise a page fault, and A/D bits tell the OS how pages are used.

Multi-Level Page Tables: Problems Solved and Trade-offs

Key Points

- **Table Size**
  Only **28 KiB** of tables, instead of **512 GiB** flat (example).
- **Contiguity**
  Each table is **one page**, placed anywhere.
- **Sparseness**
  Empty entries mean **no tables** for unmapped regions.
- **Trade-off**
  A walk needs **several memory accesses**, so it is slower.
- **Next Step**
  A translation cache, the **TLB**, comes next.

1 Multi-Level Solves the Three Problems

1. Size flat: 512 GiB	→	**28 KiB of tables** 7 tables × 4 KiB: **≈19 million× smaller**
2. Contiguity one huge block	→	**Many 4 KiB tables, anywhere** each table is **one page**; a parent entry points to it
3. Sparseness mostly empty	→	**none entry = no lower table** tables exist **only where memory is mapped**

Why 28 KiB? 48-bit VA, 4 KiB pages, 8-byte PTE (512 per table); 8 MiB mapped, contiguous

8 MiB
= 2,048 pages
→ **÷ 512 PTEs**
= **4** Level-4 tables
→ **+ 1 each**
Levels 1–3 (one 1 GiB region)
→ **7 × 4 KiB**
= **28 KiB**

2 New Problems (Trade-offs)

1. **Slower translation**
   A walk reads **one entry per level**: N reads + 1 data access.
2. **More complex HW and SW**
   Needs a hardware **page-table walker** and a cache, the **TLB**.
3. **Internal fragmentation**
   Partly used **table pages** waste memory.

Key Takeaway

Multi-level tables solve size, contiguity, and sparseness, but make translation slower, which motivates the TLB.

What Is a TLB? Hit and Miss

Key Points

- **Translation Cache**
  The **Translation Lookaside Buffer (TLB)** is a hardware cache of page translations.
- **Lookup**
  Small **L1** is searched in parallel; large **L2** is set-associative.
- **Instruction and Data**
  L1 is **split** into instruction and data TLBs; L2 is usually **unified**.
- **TLB Hit**
  The translation is found: **no page walk**.
- **TLB Miss**
  The hardware **walks the tables** and refills; a miss is **not** a page fault.

1 What Is a TLB?

A small **hardware cache** in the MMU: **VPN → PFN** plus the **PTE flags**.

PCID/ASID	VPN	PFN	Flags
address space	tag	frame	RW · U · NX · A · D · G

PCID/ASID: explained later (context switch).

Lookup by VPN: a small L1 is fully or highly associative (**CAM-like**, compared in parallel); a large L2 is **set-associative**. Details vary by CPU.

Small and fast by design: on the path of **every access**, so more entries cost **latency, power, area**.

L1 is usually split: **iTLB** (instruction fetch) and **dTLB** (load / store). **L2 is usually unified.**

Search sequence: VA → **L1 TLB** → (miss) **L2 TLB** → (miss) **page walk**. A hit returns the PA; L2 hits fill L1.

2 TLB Hit

Hit: PA = **PFN + offset**. Permissions checked from the **cached flags**. **No table read.**

The translation is **overlapped with the L1 cache access**, so a hit adds almost **no extra latency**.

3 TLB Miss

Miss: the **hardware walker** reads the page tables.
Valid → fill the TLB, retry.
Invalid → page fault (OS).

A miss is **not** a fault. A hit can still fault (**permission violation**).

Key Takeaway

The TLB is a small, fast, parallel-lookup cache of translations: a hit skips the walk, a miss walks and refills.

TLB Issue: TLB Reach

Key Points

- **TLB Reach**
  Reach = **entries × page size**: memory mapped at once.
- **Small Reach**
  Only **KiB to MiB**, while RAM is GiB.
- **Working Set**
  The pages a program **actively uses** in a time window.
- **Fits or Not**
  If it fits, **most accesses hit**; if not, **capacity misses** occur.
- **Remedies**
  Use **larger pages**, better **locality**, or faster misses.

1 TLB Reach

Reach = #entries × page size

Example	#entries	Page size	Reach
L1 TLB	64	4 KiB	256 KiB
L2 TLB	2,048	4 KiB	8 MiB

The memory the TLB can map **at once**. Only **KiB–MiB**, while RAM is **GiB**.

2 Working Set

The **working set**: the pages a program **actively uses** in a time window.

2 MiB working set < 8 MiB reach: **fits**

1 GiB working set > 8 MiB reach: **does not fit**

3 Reach vs. Working Set

Working set ≤ reach: entries stay resident: **mostly hits**.

Working set > reach: evicted before reuse: **capacity misses**.

Random over 1 GiB: hit rate ≈ 8 MiB / 1 GiB ≈ **0.8%** (ideal): nearly every access misses.

Sequential: one miss per 4 KiB page, then hits.

4 How to Stay Within Reach

1. Raise reach: **larger pages** (**huge pages**, later): one entry maps 2 MiB or 1 GiB.

2. Shrink the working set: **locality**: pack hot data together, process data in blocks, avoid sparse pointer chasing.

3. Hardware helps: **L2 TLB**, **walk cache**, caches for PTEs: make misses cheaper.

Key Takeaway

TLB reach is small: when the working set exceeds it, entries are evicted before reuse and capacity misses dominate; fix it with larger pages or better locality.

TLB Issue: TLB Shootdown

Key Points

- **Per-core TLB**
  Each core has its **own TLB**.
- **No Coherence**
  Hardware does **not** keep TLBs consistent with page tables or each other.
- **Stale Entry**
  Another core may still use a **freed frame**.
- **TLB Shootdown**
  The OS **invalidates** entries on the other cores.
- **Cost**
  The cost grows with cores, so the kernel **batches** flushes.

1 Per-Core TLB, No Coherence

Yes: **each core has its own TLB** (L1 and L2).

	Per core?	Kept coherent by HW?
Data cache	yes	**yes** (coherence protocol)
TLB	yes	**no**: not with page tables, not with other TLBs

A core keeps whatever it **cached**. Changing a PTE in memory **does not update or invalidate** TLB entries.

2 Why It Is a Problem

Threads of one process run on **Core 0** and **Core 1**, sharing the page table.

Core 0	Core 1
munmap(): PTE cleared, page freed	TLB still has VPN → PFN

Core 1 can still access the **freed frame**, now reused by **another process**: data corruption or a **security hole**.

3 TLB Shootdown

1. update PTE
2. flush own TLB
3. IPI to cores running this process
4. they invalidate, ack
5. initiator waits, then frees page

TLB shootdown: software invalidates remote TLBs. **x86**: inter-processor interrupt (**IPI**). **Arm64**: broadcast **TLBI** instruction, no IPI.

Cost: interrupts, waiting, grows with cores. Linux limits it to cores running the process and **batches** flushes.

Key Takeaway

TLBs are per core and not coherent: after changing a PTE, the OS must run a TLB shootdown before reusing the page.

TLB Issue: Context Switching

Key Points

- **Same Page Number**
  Different processes can use the **same page number** for different frames.
- **Address Space IDs**
  PCID (x86-64) and **ASID** (ARM64) tag TLB entries.
- **Without Flush**
  Tagged entries **stay cached** across a context switch.
- **Without Tags**
  Switching address spaces generally needs a **TLB flush**.
- **Global Entries**
  Kernel entries have **no ID** and are **not flushed**.

1 Two Processes, Same Virtual Page Number

Process A PCID/ASID = 7	→	**VPN = 0x400**	→ **PFN = 100**
Process B PCID/ASID = 12	→	**VPN = 0x400**	→ **PFN = 250**

Same VPN, different frames.
A stale translation must never be used for another address space.

2 TLB Entries Tagged by ID

ID	VPN (tag)	PFN	Process
7	0x400	100	A
12	0x400	250	B

Lookup key = **identifier + VPN**. Same VPN can have different PFNs.

3 Without vs. With IDs

Without PCID/ASID
Switching address spaces needs a **TLB flush** (invalidate old entries).

With PCID/ASID
Entries are **tagged**; different processes **coexist**, fewer flushes.

4 Global Entries: Kernel

Kernel mappings are the **same in every address space**: they need **no ID**.

Kernel: Global
x86 **G=1** (Arm64 **nG=0**): matches **any** PCID/ASID, **not flushed** on a switch.

User: tagged
x86 **G=0** (Arm64 **nG=1**): tagged with **PCID/ASID**, coexist.

Key Takeaway

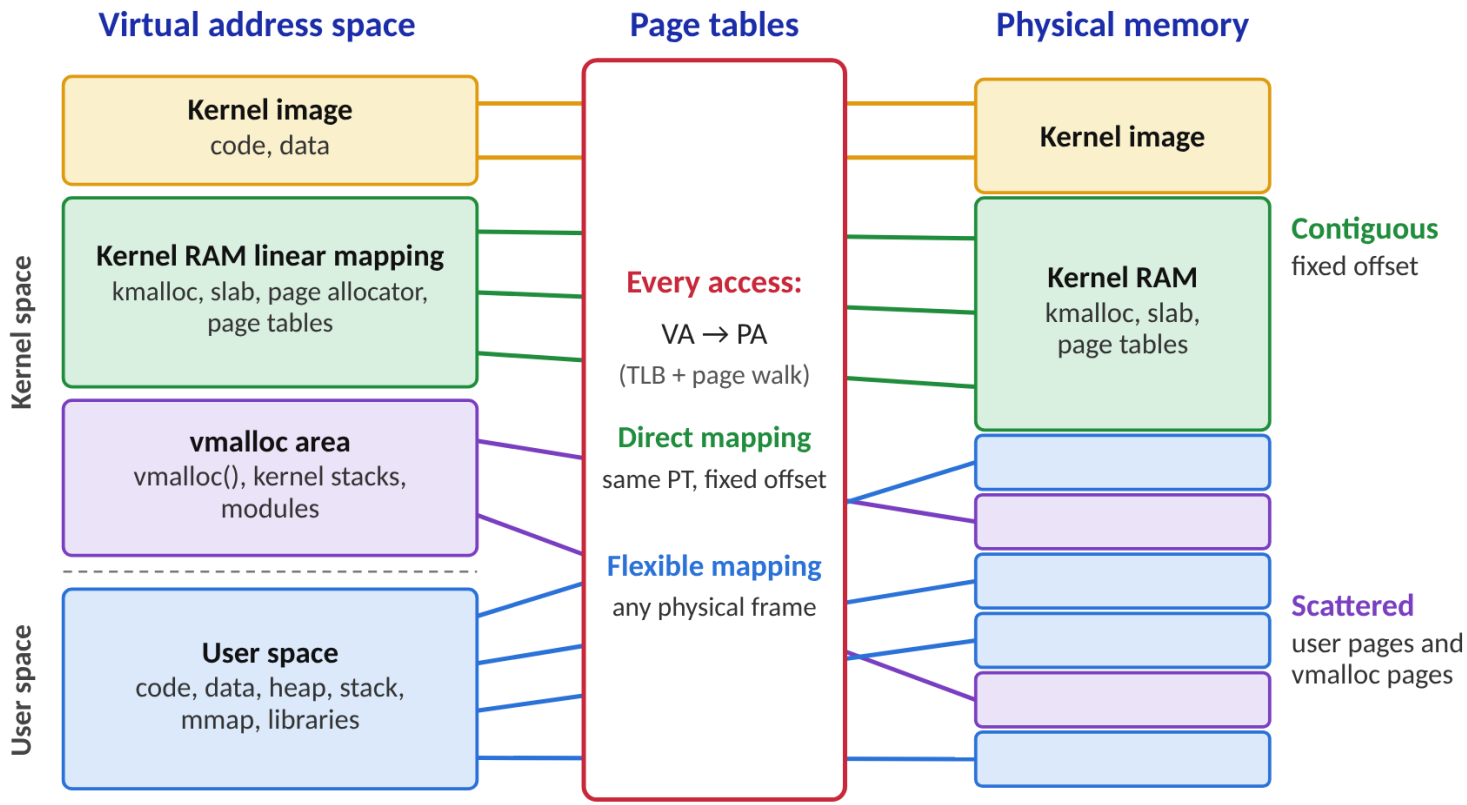
PCID (x86-64) and ASID (ARM64) let TLB entries from different address spaces coexist, reducing flushes; shared kernel entries are Global and need no ID.

Kernel and User Mappings: What Is Mapped Where

Key Points

- **Always Virtual**
  Every access uses a **virtual address**.
- **User Space**
  Each process has its own page tables; frames come **on demand**.
- **RAM Mapping**
  The kernel sees all RAM at **virtual = physical + constant**.
- **Kernel Image**
  Kernel code and static data are **contiguous**.
- **vmalloc Area**
  Virtually contiguous, but physically **scattered**.

Virtual Address Space → Page Tables → Physical Memory



Key Takeaway

Every address is virtual: the RAM mapping and kernel image are contiguous with a constant offset, while vmalloc and user space map scattered frames.

Support for Multiple Page Sizes

Key Points

- **Three Page Sizes**
  Linux supports **4 KiB, 2 MiB, and 1 GiB** pages, chosen per mapping.
- **Early Walk End**
  With **PS = 1** at the PUD or PMD level, the walk ends early.
- **Address Calculation**
  Physical address = **physical base + page offset**.
- **Kernel Usage**
  Large pages for the **RAM mapping**; 4 KiB for vmalloc.
- **User Usage**
  Mostly **4 KiB**; larger via **THP** or **hugetlb**.

1 Page Sizes and Where the Walk Ends

Page size	Where the walk ends	Offset bits	Used by
1 GiB	PGD → PUD **(PS = 1)**	30	RAM linear mapping, hugetlb
2 MiB	PGD → PUD → PMD **(PS = 1)**	21	RAM linear mapping, kernel image, THP, hugetlb
4 KiB	PGD → PUD → PMD → **PTE**	12	vmalloc, user (default)

PS = 1: the walk ends at that level. Physical address = **physical base + page offset**.

2 How User Programs Get Huge Pages

Default: 4 KiB

- Most user memory uses **4 KiB** pages.

THP: 2 MiB

- **Admin sets the mode** (sysfs):
  always: kernel uses THP automatically
  madvise: only where the app calls **madvise(MADV_HUGEPAGE)**
- **Best-effort**: falls back to 4 KiB
- **Swappable** (split first)

hugetlb: 2 MiB / 1 GiB

- **Admin reserves a pool** (vm.nr_hugepages)
- App asks with **mmap(MAP_HUGETLB)**
- **Guaranteed**: no fallback
- **Never swapped**

Key Takeaway

One page table supports several sizes: the walk ends at PUD, PMD, or PTE, and each mapping picks its size.

Why Multiple Page Sizes? Benefits and Trade-offs

Key Points

- **Large Pages**
  Fit a **hot, large** working set: fewer TLB misses.
- **Small Pages**
  Fit **small or sparse** memory: little waste.
- **Fewer Entries**
  One 1 GiB entry replaces **262,144** small ones.
- **TLB Reach**
  Fewer translations cover **more memory**.
- **Trade-off**
  Coarser **granularity** and higher allocation cost.

1 Why Choose Large or Small?

Large pages: 2 MiB / 1 GiB
Best for a **large, hot working set** (DB buffers, JVM heap, VM guests).
Gain: fewer TLB misses, shorter walks, fewer page faults.
Cost: wasted memory, slower first touch.

Small pages: 4 KiB
Best for **small or sparse** memory and many processes.
Gain: little waste; easy to allocate, share, swap.
Cost: more TLB entries and walks.

2 Fewer Translations

Page size	Leaf entries to map 1 GiB
4 KiB	262,144
2 MiB	512
1 GiB	1

Translations needed to cover the region, not actual TLB entry counts.

3 Benefit and Trade-off

Fewer translations cover the same memory: **more TLB reach**, fewer TLB misses, shorter walk.

Trade-off: huge pages improve TLB efficiency but **reduce management granularity** and may **increase allocation costs**.

Key Takeaway

Large pages need fewer translations and give more TLB reach, but cost granularity and allocation time.

Why the Kernel Runs Fast: The Mapping View

Key Points

- **Prebuilt Mapping**
  Kernel RAM is **always mapped**: no demand faults.
- **Large Pages**
  2 MiB or 1 GiB entries give more reach and a shorter walk.
- **Global Entries**
  Kernel TLB entries **survive** context switches.
- **Exceptions**
  vmalloc and split mappings cost more.
- **Not Guaranteed**
  Speed comes from the **mapping layout**; the TLB is still finite.

1 Why Kernel Mappings Are Cheap

Design choice	What it removes	Mechanism
Prebuilt mapping	**No demand faults** on kernel RAM	Always mapped, built at boot
Large pages	Fewer TLB entries, **shorter walk**	1 GiB / 2 MiB entries
Global entries	Kernel TLB entries **survive context switches**	x86 G bit / ARM64 nG = 0
Shared tables	Walk hits **cached PTEs**: cheaper page walks	One kernel mapping, all processes

2 Where the Cost Comes Back

vmalloc, stacks, modules
4 KiB pages, more TLB entries

Split mappings
set_memory_ro() breaks a large page into 4 KiB

TLB shootdown
IPIs to flush other CPUs

3 Kernel Design, Not Hardware

Not guaranteed by hardware. The MMU treats kernel and user addresses alike; both share one TLB, tagged by **PCID / ASID**.

Speed comes from how the kernel lays out its mappings: prebuilt, large, global, shared.

Finite TLB: a busy workload still evicts kernel entries.

Key Takeaway

The kernel is fast because its mappings are prebuilt, large, and global: the hardware does not guarantee it, and vmalloc and split mappings pay extra.

Observing TLB and Page Walks

Key Points

- **Observation**
  Count TLB misses and page walks with **performance counters** via **perf**.
- **Event Names**
  Names are **architecture-dependent**; check **perf list**.
- **Measured Result**
  Access time **steps up** as the working set outgrows each TLB level.
- **Huge Pages**
  Huge pages can reduce TLB misses.
- **Tuning**
  Measuring helps you **understand and tune** performance.

1 Find the PMU Events

x86-64 (examples)

```
$ perf list | grep tlb
dTLB-load-misses
dTLB-store-misses
iTLB-load-misses
dtlb_load_misses.walk_completed
```

ARM64 (examples)

```
$ perf list |
grep tlb
L1D_TLB_REFILL
L2D_TLB_REFILL
DTLB_WALK
ITLB_WALK
```

Event names are **architecture-dependent**; use **perf list** on your system.

2 Measure with perf stat

```
$ perf stat \
-e dTLB-load-misses \
-e dtlb_load_misses.walk_completed \
./a.out
```

Counts **misses** and **completed walks**. Needs **PMU access**: many VMs expose none.

3 Measured: Access Time vs. Working Set

This CPU (inferred from the steps): the **L1 TLB** covers 64 pages (256 KiB); the **L2 TLB** covers 1,024 pages (4 MiB) but not 4,096 (16 MiB).

4 KiB pages	Working set	ns / access	Translation served by
64	256 KiB	2.0	L1 TLB hit
1,024	4 MiB	4.3	L2 TLB hit
4,096	16 MiB	11	page walk
32,768	128 MiB	18	page walk

Steps appear where the working set passes each reach. Each access hits a different page, but all pages share **one physical page**, so data stays in the L1 cache: only translation changes. Intel Xeon, KVM VM; values vary by CPU.

Key Takeaway

TLB misses and page walks can be measured with perf; access time steps up as the working set outgrows each TLB level.

Checkpoint: Virtual to Physical Mapping

Checkpoint: Key Points

- **Page Table**
  Multi-level tables solve **size** but cost **walk speed**.
- **Miss or Fault**
  A **TLB miss** causes a walk; a **not-present** entry causes a fault.
- **Reach and Shootdown**
  Reach = entries × page size; **stale entries** need shootdown.
- **Context Switch**
  PCID / ASID tags and **global** kernel entries avoid flushes.
- **Page Sizes**
  Larger pages **end the walk early**: THP and hugetlb.

1 Multi-Level Page Table

Q: What is solved, and what is the issue?

Solved: size. Flat **512 GiB** vs. multi-level **16 KiB** for one mapped page.

Issue: speed. Every miss needs **4 memory reads** (page walk).

2 Miss or Fault?

Q: Is every TLB miss a page fault?

Miss + **Present** PTE: hardware **page walk**.

Present = 0 or permission violation: **page fault**. A hit can still fault.

3 TLB Reach

Q: What is TLB reach, and why is it important?

Reach = **entries × page size**.

Working set above reach: **misses and walks**. **2 MiB** pages multiply reach by **512**.

4 Shootdown

Q: Why do we need a shootdown?

TLBs are **per core**, not coherent: other cores may keep a **stale entry**.

Update PTE, flush own, **IPI**, ack, then free the page.

5 Context Switch

Q: How can a switch skip the TLB flush?

Entries are tagged **PCID / ASID**: each process hits only its own.

Kernel entries are **global**: they stay across switches.

6 Page Sizes

Q: How do we support multiple page sizes?

End the walk early: 2 MiB at **PMD**, 1 GiB at **PUD** (PS = 1).

THP: best-effort, swappable. **hugetlb**: reserved, never swapped.

Key Takeaway

A TLB miss costs a walk, not a fault; reach, shootdowns, tags, and page size decide how often the walk happens.

PART 3: Memory Management

- **Week 8**

- **Address Translation, Paging, and TLB**
  - How does a virtual address translate into a physical memory access?
- **Page Fault and Physical Page Allocation**
  - How does Linux obtain and map a physical page when a page fault occurs?
- **Buddy Allocator, SLUB, and Kernel Object Allocation**
  - How does Linux efficiently allocate and manage physical pages and kernel objects?

What Causes a Page Fault?

Key Points

- **Classified into two types**
  By the PTE **P bit**: a **not-present fault** (P = 0) or a **permission fault** (P = 1).
- **Minor page fault**
  Resolved **without storage I/O**: the page is already in memory, or a new zeroed page is enough.
- **Major page fault**
  The kernel must **read the page from storage** (file or swap), so it is typically **orders of magnitude slower**.

1 Not-present Fault

P = 0: no valid mapping exists

P	RW	U	NX
0	–	–	–

Typical causes
- No PTE exists (nothing mapped yet)
- Demand paging (first touch)
- File page not yet mapped
- Swapped-out page

2 Permission Fault

P = 1: mapping exists, but the access is not allowed

P	RW	U	NX
1	0/1	0/1	0/1

Typical causes
- Write to a read-only page
- User access to a kernel page
- Fetch from an NX page

3 Minor Fault vs. Major Fault

Minor Fault

Resolved **without storage I/O**.
- The page is already in memory, or a **new zeroed page** is enough
- **Cost**: low

Major Fault

Needs **storage I/O** (file or swap).
- The page must be **read from storage**
- **Cost**: typically **orders of magnitude** slower than minor

Key Takeaway

A page fault is not present (P = 0) or a permission violation (P = 1); a not-present fault is minor or major depending on storage I/O.

Not-present Faults: Anonymous Pages

Key Points

- **Anonymous pages**
  Memory with **no file behind it**: heap (malloc), stack, and anonymous mmap.
- **First touch (minor)**
  No PTE yet. The kernel first adds any **missing page-table pages** (one frame each), then a **zero-filled frame** for the data.
- **Swapped-out (major)**
  The PTE holds a **swap entry**. Reading the content back from swap needs **storage I/O**; a **swap cache hit** avoids it and stays minor.

1 First Touch — Minor

Fault
PTE: none (never mapped); VMA is anonymous

↓

Page-table pages
a missing level (PUD / PMD / PTE) gets **its own zeroed frame**

↓

Get a data frame
write: new **zero-filled** frame
read: shared **zero page**

↓

Set the PTE
PTE: none → **PFN**; retry the access

2 Swapped-out Page — Major

Fault
PTE: swap entry (content is on swap)

↓

Page tables
already exist (the PTE holds the swap entry); **never swapped out**

↓

Get a frame, read swap
swap-in from the swap device

↓

Set the PTE
PTE: swap entry → **PFN**; retry the access

Key Takeaway

An anonymous page is created on its first touch (minor) or restored from swap (major).

Not-present Faults: File-backed Pages

Key Points

- **File-backed pages**
  Memory backed by a file: executables, shared libraries, and **mmap**ed files. The data lives in the **page cache**.
- **Cache hit (minor)**
  The page is already cached, so the kernel only **maps the cached frame**. No I/O is needed.
- **Cache miss (major)**
  The kernel must **read the page from storage** into the page cache first, then map it.
- **Private write (COW)**
  If the **first access is a write** to a MAP_PRIVATE mapping, one fault **copies** the file page.

1 Cache Hit — Minor

Fault
PTE: none; VMA is **file-backed**

↓

Look up the page cache
the page is **found**

↓

Set the PTE
map the **cached frame**: no new data frame, no I/O; **fault-around** maps nearby cached pages too

2 Cache Miss — Major

Fault
PTE: none; VMA is **file-backed**

↓

Look up the page cache
not found

↓

Read from storage
new frame in the page cache; **read-ahead**

↓

Set the PTE
map the new **cached frame**

3 Private Write — Minor/Major

Fault
first access after exec() is a write; PTE: none (e.g. **.data**)

↓

Get the file page
hit: minor; miss: read (major)

↓

Copy it (COW)
into a **new anonymous frame**

↓

Set the PTE
map the private copy, writable

Read first, write later?
The PTE exists (P = 1), so the later write is a **permission fault**.

Key Takeaway

A file-backed fault maps a page-cache frame: minor if it is already cached, major if it must be read from storage.

Not-present Faults: Other Cases

Key Points

- **Stack growth**
  The kernel **extends the stack VMA** on demand, then handles it like a first touch.
- **Shared memory**
  tmpfs, memfd, and System V shared memory are **RAM-backed**: minor on first touch, major after swap-out.
- **Invalid address**
  With **no VMA**, the kernel sends **SIGSEGV** and allocates nothing.
- **Kernel-mode fault**
  A system call may touch a user buffer that is **not present**. The kernel handles it like a user fault; a bad address returns **-EFAULT**.

1 Stack Growth — Minor

- The address is just **below the stack** VMA
- The kernel **grows the VMA**, within the stack limit
- Then it is an anonymous first touch

2 Shared Memory — Minor/Major

- File-like, but backed by **RAM** (and swap)
- First touch: new **zeroed page** in the shmem cache
- After swap-out: **swap-in** (major)

3 Invalid Address — SIGSEGV

- The address is in **no VMA** (NULL, freed memory)
- Nothing to map: **no page** is allocated
- The process gets **SIGSEGV**

4 Kernel-mode Fault — Kernel

- A system call (e.g. **read()**) accesses a **user buffer** via **copy_to_user**
- The page is **not present**: the fault occurs in **kernel mode**
- Valid: handled like a user fault. Invalid: **-EFAULT**

Key Takeaway

Not every not-present fault maps a page: the kernel may grow the stack, use RAM-backed shared memory, or end with SIGSEGV.

Permission Faults: Permission Bits

Key Points

- **Permission fault**
  The mapping exists (P = 1), but a **PTE permission bit** denies the access.
- **Hardware Reports**
  The MMU only reports the fault; it does not know the **VMA** policy.
- **Kernel Decides**
  The kernel checks the **VMA**: if it denies the access, the process gets **SIGSEGV**.
- **Legitimate Cases**
  If the VMA allows it, such as **COW**, the kernel **fixes the PTE** and retries.

Permission Fault: Which Bit Denied It?

P = 1: the mapping exists, but a PTE permission bit denies the access: **RW = 0** and a write, **U = 0** and a user-mode access, or **NX = 1** and an instruction fetch (see the PTE slides).

Kernel Decision: Is It Really an Error?

Permission fault (#PF, P = 1)
The kernel looks up the **VMA** of the address and checks the access.

↓ VMA denies

Invalid access
The kernel sends **SIGSEGV** to the process.

↓ VMA allows

Legitimate: fix and retry
The kernel **fixes the PTE** (and may copy the page); the CPU retries.

Legitimate Permission Faults

COW after fork
Parent and child share the page **read-only**. A write faults; the VMA allows writes, so the kernel **copies the page**.

Write after read first
A read maps the shared **zero page** read-only. The later write faults; the kernel allocates a **new frame**.

Key Takeaway

A permission fault means the PTE denies the access; it is usually SIGSEGV, but not when the VMA allows it, as in COW.

Summary of Page Fault Handling Flow

Key Points

- **Exception**
  A page fault enters the **kernel** through an exception (**#PF**). CR2 holds the faulting address.
- **Validate**
  The kernel checks the access against the **VMA**. If the VMA does not allow it, the process gets **SIGSEGV**.
- **Resolve**
  Otherwise the kernel **makes the page available**: map it, allocate it, read it, or fix the PTE.
- **Resume**
  The **PTE is updated** and the instruction is **retried**. Most faults are normal, not errors.

Page Fault Handling Steps

1 Exception
- CPU traps (#PF)
- CR2 = fault address
- Error code saved

2 Validate
- Find the **VMA**
- Access allowed?
- No: SIGSEGV

3 Resolve
- Make the page available (see below)

4 Update PTE
- Install PFN and permission bits
- Flush TLB when a mapping is replaced (COW)

5 Resume
- Return from the handler
- CPU retries the instruction

Resolve: by Case

Case	Action
Not present: minor	Map or allocate; no I/O
Not present: major	Read from storage
Permission: allowed	Fix the PTE (e.g. COW)
Invalid access	SIGSEGV

Normal vs. Error

Resolved: normal
Most page faults end here: the kernel fixes the PTE and the CPU **retries**.

Invalid: the only error
The kernel sends **SIGSEGV**; the process is not resumed.

Key Takeaway

A page fault traps to the kernel, which validates the address, resolves the fault, updates the page table, and retries the instruction.

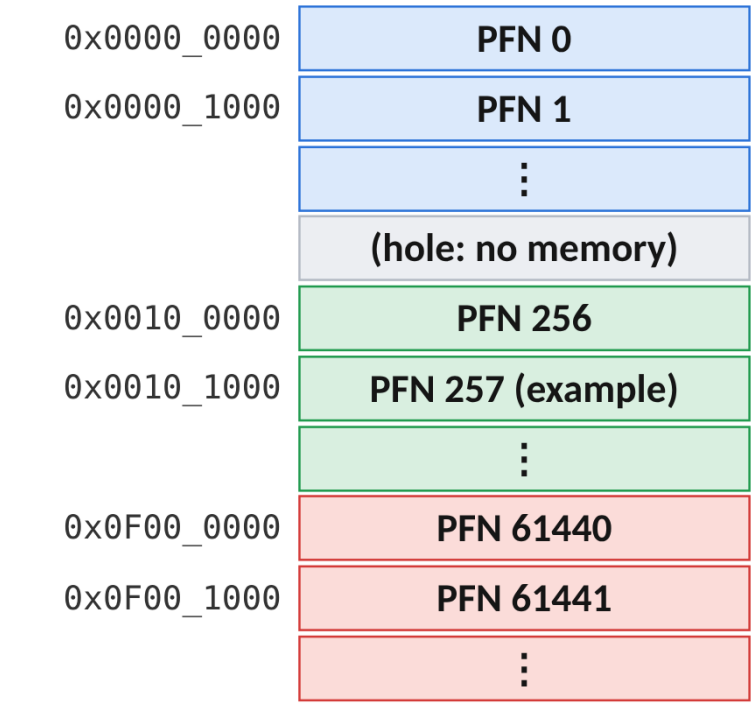
Physical Memory: Page Frames and PFN

Key Points

- **Page Frame**
  Physical memory is divided into **fixed-size frames** (PAGE_SIZE, typically 4 KiB). A huge page = many frames.
- **Page Frame Number**
  Each frame has a unique **PFN = PA >> 12**.
- **Offset in a Frame**
  The **low 12 bits** of a PA are the offset in the frame.
- **Page Frame State**
  Kept in **struct page** (later slide).
- **Bit Width**
  PFN = PA bits − 12: **52** in theory, **40 / 36** by arch limit.

Each page frame has a PFN; a physical address = **PFN + offset**.

Physical Address Space: 4 KiB Page Frames



Free: available
Allocated: in use
Reserved: device, firmware, kernel
Hole: no memory behind it

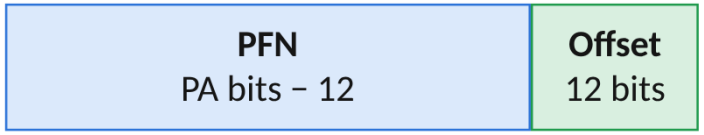
PFN and Offset

PFN = PA >> 12
PA = (PFN << 12) | offset
offset = PA & 0xFFF

PA `0x0010_1ABC` → PFN **0x101** + offset **0xABC**

Bits	PA	PFN	Offset
Theory (64-bit)	64	**52**	12
x86-64 limit	52	**40**	12
arm64 limit	48 (52)	**36** (40)	12

Physical address layout



e.g. x86-64: PA 52 bits → PFN **40** bits + offset **12** bits

Key Takeaway

Physical memory is split into fixed-size page frames; PFN = PA >> 12, and the low 12 bits are the offset.

Struct Page: Physical Page Metadata

Key Points

- **Fixed-size Pages**
  Linux manages memory in **fixed-size pages** (e.g., 4 KiB).
- **struct page**
  Each physical page has a **struct page** holding its metadata.
- **Finding struct page**
  Indexed by PFN: **mem_map[pfn]**.
- **Kernel Users**
  Buddy allocator, page cache, SLUB, and reclaim use it.
- **Layout Changes**
  Exact layout varies by kernel version; the **role is the same.**

The kernel keeps one **struct page** per physical page frame to track its state, mapping, and usage.

One struct page per Page Frame

	Physical memory page frames	1 : 1	mem_map[] struct page array
PFN 0	4 KiB	↔	struct page
PFN 1	4 KiB	↔	struct page
PFN 2	4 KiB	↔	struct page
	⋮	↔	⋮
PFN N	4 KiB	↔	struct page

`pfn_to_page(pfn)` → `&mem_map[pfn]`

`page_to_pfn(page)` → `page - mem_map`

What struct page Stores

- **Page state / flags**: free, allocated, active, dirty, ...
- **Reference count**: how many users reference the page (page tables, page cache, slabs)
- **Mapping info**: which address space or object uses it (anonymous, page cache, file)

How It Is Used

- **Buddy allocator**: manage free and allocated pages
- **Page cache**: track file-backed pages
- **SLUB**: manage slab pages
- **Reclaim / LRU**: track page usage

Key Takeaway

struct page is the kernel's metadata for each physical page, used to manage, allocate, and reclaim memory.

Physical Memory Zones

Key Points

- **Memory Zones**
  Linux divides physical memory into **zones**.
- **Zone Contents**
  Each zone holds **page frames** with similar constraints.
- **Why Zones**
  Devices and kernel subsystems have different needs (**DMA**, addressability).
- **Zone Selection**
  The kernel picks a zone from the allocation **flags** (e.g., GFP_DMA).
- **Zone Variation**
  Zone setup varies by **architecture and kernel version**.

Zonal Layout (x86-64, typical)

Zones split by **address range** and usage constraints.

0x0000_0000	**DMA** 0 – 16 MB	Legacy DMA devices: low addresses only
0x0100_0000	**DMA32** 16 MB – 4 GB	32-bit DMA devices (≤ 4 GB)
0x1_0000_0000	**Normal** > 4 GB	General-purpose memory

Why Zones Are Needed

1. Device DMA limits
some devices reach only low physical addresses (< 16 MB or < 4 GB)

2. Allocation policy
keep pages with different constraints separate

3. Fragmentation control
free lists per zone improve allocation success

Common Zones

Zone	Range	Used for
`DMA`	0–16 MB	Legacy DMA devices
`DMA32`	16 MB–4 GB	32-bit DMA (many PCIe)
`Normal`	> 4 GB	General-purpose memory

Architecture Differences

Architecture	Typical zones
`x86-64`	DMA, DMA32, Normal (no HighMem)
`ARM64`	Often DMA, DMA32, Normal; sizes vary

Key Takeaway

Linux groups pages into zones because devices and kernel components have different addressability and allocation requirements.

Page Frame Allocation after a Page Fault

Key Points

- **New Frame Needed**
  Only faults that **create a page** need a new frame; a **cache hit** does not.
- **Page Allocator**
  The **page allocator** supplies frames (details in the next part).
- **Batch Allocation**
  Batching and **huge pages** cut allocation work.
- **Frame Shortage**
  Reclaim and the **OOM killer** handle a shortage.

Needs a New Frame

- **Anonymous first touch** (incl. stack, shmem): allocate a zeroed frame
- **COW**: allocate a frame for the copy
- **Page cache miss**: allocate a frame in the page cache
- **Swap-in**: allocate a frame in the swap cache
- **Page-table pages**: allocate PUD / PMD / PTE pages

No New Frame

- **Page cache hit**: map the cached frame
- **Swap cache hit**: map the cached frame
- **Anonymous read first touch**: map the shared read-only zero page; a later write allocates a frame (COW)

Allocation Path

Fault handler needs a frame → **Get a free frame** from the allocator → **Fill the frame** zero / copy / read → **Set the PTE** PFN + VMA permissions

Several Frames at Once

- **Allocator** hands out blocks of 2^order frames
- **Read-ahead** adds many pages to the page cache
- **Fault-around** maps ~16 cached pages (no new frame)
- **THP** allocates 512 frames at once

When Free Frames Run Out

- Below the watermark: **reclaim** frees frames (Week 9)
- **cgroup** limit: reclaim inside the cgroup (Week 9)
- Still short: **OOM** (Week 10)

Key Takeaway

A page fault that needs a new page asks the page allocator for a frame; when none is free, reclaim takes over.

Observing Minor / Major Page Faults

Key Points

- **Observation Tools**
  Observe faults with **perf**, **/proc**, or **time**.
- **Minor Page Fault**
  Resolved **without storage I/O**: first touch, cache hit, COW.
- **Major Page Fault**
  Needs **storage I/O** (disk or flash).
- **Access Patterns**
  Different access patterns give **different fault counts**.

1 Reading the Counters

Counter	Counts
`page-faults`	all faults (minor + major)
`minor-faults / minflt`	resolved without storage I/O
`major-faults / majflt`	needed storage I/O

Check: page-faults = minor + major

2 Measuring Page Faults

```
$ perf stat -e page-faults ./alloc_touch
  25656 page-faults (25656 minor, 0 major)
$ cat /proc/<pid>/stat  # minflt, majflt
$ /usr/bin/time -v ./alloc_touch
```

perf is recommended; **/proc** and **time** give the same counters.
Example environment: Xeon 2.8 GHz cloud VM, Linux 6.x, 4 KiB pages, virtual disk. Values vary by version and environment.

3 Minor Faults: Allocate vs. Touch

alloc_only = malloc(100 MB), no access
alloc_touch = + write 1 byte per 4 KiB page

```
alloc_only:    57 minor, 0 major
alloc_touch: 25656 minor, 0 major
```

Observe: 100 MB / 4 KiB = 25,600 pages. The difference, 25,656 − 57 = 25,599, is one minor fault per first-touched page. The 57 are start-up faults (program, libraries).

4 Major Faults: Cold vs. Warm

Task Touch every page of a 64 MB file (16,384 pages). Cold = page cache emptied first.

Case	Major faults	Elapsed
Cold, read-ahead off	**16,384**	1.16 s
Warm (in page cache)	**0**	0.010 s
Cold, read-ahead on	**1**	≈ 0.1 s

Observe: read-ahead off (MADV_RANDOM): one major fault per page. Read-ahead on: pages are read in advance, so one I/O serves many. **Warm is ≈ 10× to 100× faster than cold.**

Key Takeaway

Minor faults need no storage I/O; major faults do. Counting them explains app start-up and memory use on webOS TV.

Checkpoint: Page Fault vs Physical Page Allocation

Key Points

- **Fault vs. Allocation**
  A page fault does **not always allocate** a new frame.
- **Minor or Major**
  It is **major** only when the kernel must wait for **storage I/O**.
- **New Frame Allocation**
  A new frame is needed when a **page is created or copied**: first touch, COW, cache miss, swap-in.
- **Mapping Only**
  If the page is already in memory, the kernel just **maps** it.

For each scenario: what is the **P bit**, is it **minor or major**, and is a **new frame** allocated?

#	Scenario	P bit	Type	New frame?	Why
1	Write to a never-touched anonymous page	0	Minor	Yes	Demand allocation: a **new zeroed frame**
2	Read a never-touched anonymous page	0	Minor	No	Maps the shared **zero page**
3	File page already in the page cache, not yet mapped	0	Minor	No	Maps the **cached frame**
4	File page not in the page cache	0	Major	Yes	**Reads from storage** into a new page-cache frame
5	Swapped-out anonymous page	0	Major	Yes	**Reads from swap** into a new frame
6	First write to a private file mapping (e.g. .data) after exec()	0	Minor / Major	Yes	Gets the file page, then **copies** it (COW)
7	Write to a page that was read first (zero page or shared)	1	Minor	Yes	**Permission fault**, VMA allows: copy (COW)
8	Write to a read-only region (e.g. code)	1	SIGSEGV	No	VMA denies: the process gets **SIGSEGV**

Key Takeaway

Minor or major depends on storage I/O; a new frame is needed only when a page must be created or copied.

PART 3: Memory Management

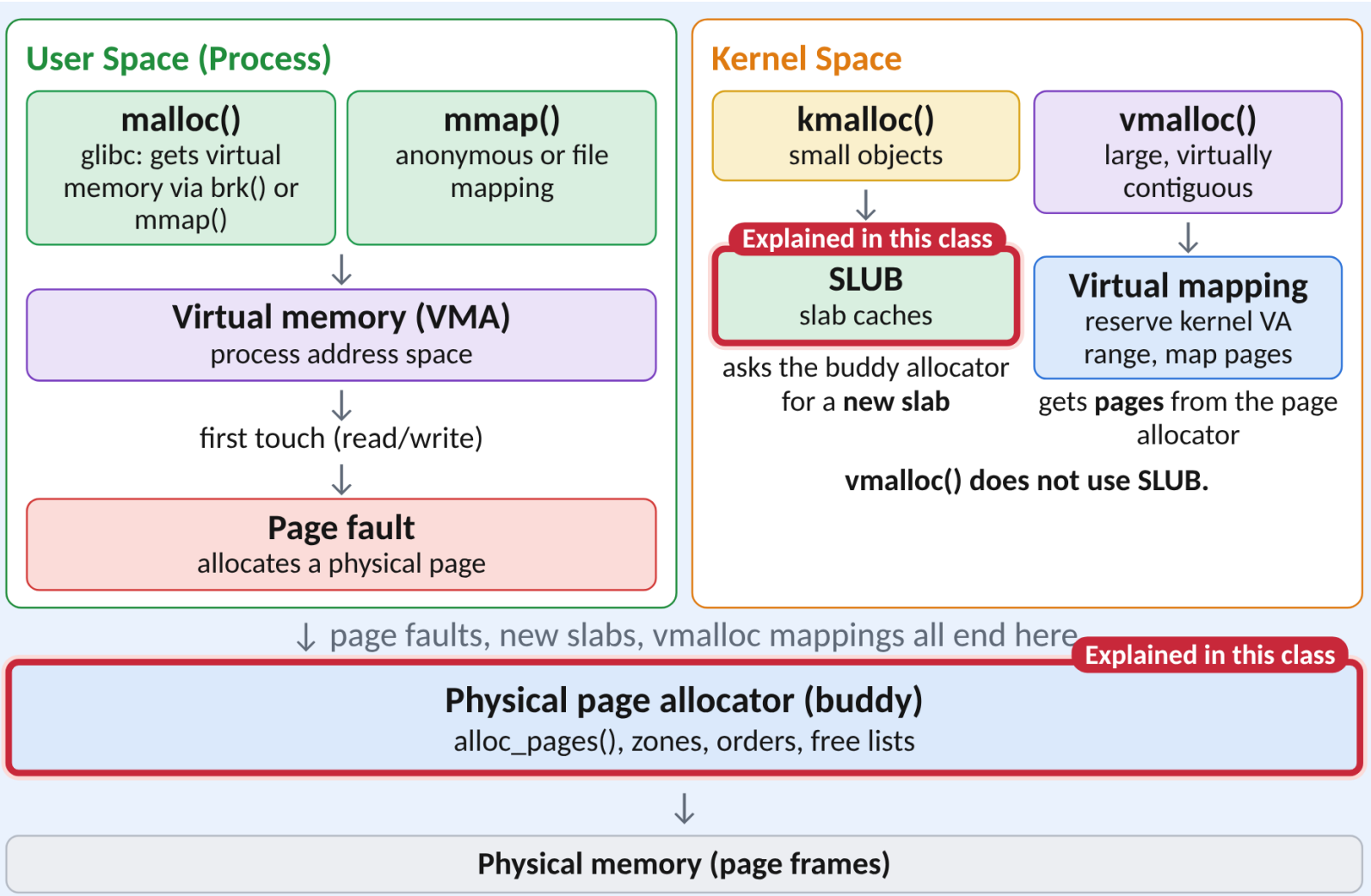
- **Week 8**

- Address Translation, Paging, and TLB
  - How does a virtual address translate into a physical memory access?
- Page Fault and Physical Page Allocation
  - How does Linux obtain and map a physical page when a page fault occurs?
- **Buddy Allocator, SLUB, and Kernel Object Allocation**
  - How does Linux efficiently allocate and manage physical pages and kernel objects?

Linux Kernel Memory Allocators: Landscape

Key Points

- **Different allocators**
  Linux uses different allocators for **different purposes**.
- **User space**
  malloc() gets virtual memory; pages arrive on demand.
- **Kernel**
  kmalloc() uses SLUB; **vmalloc()** maps scattered pages (not via SLUB).
- **Common base**
  All end at the **buddy allocator**: we study it first, then **SLUB**.



Key Takeaway

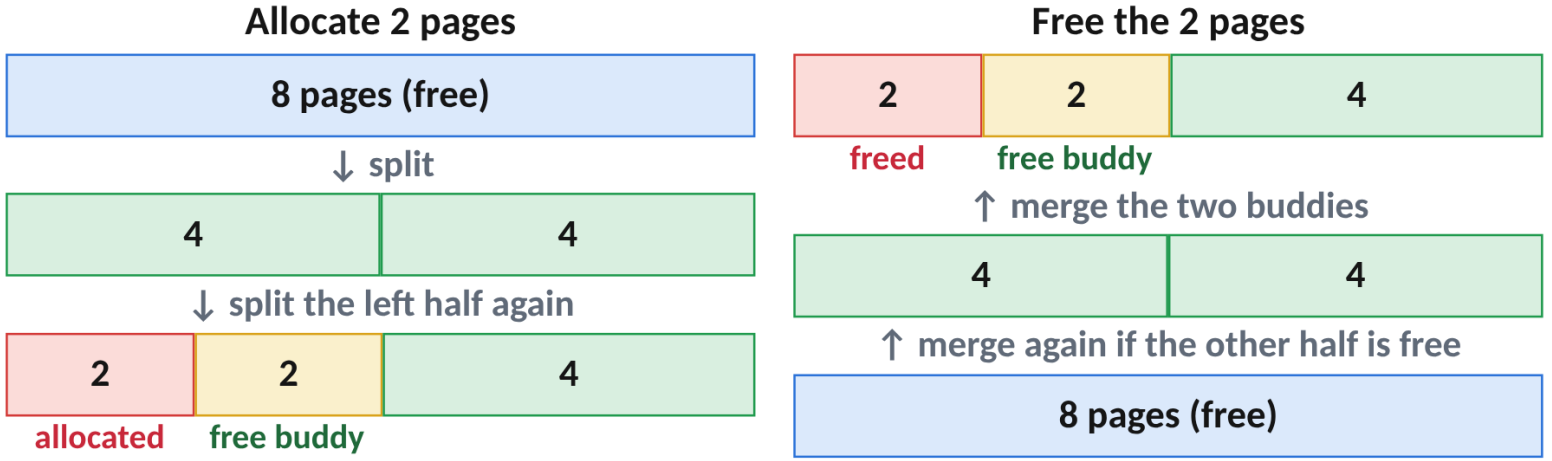
User malloc() gets pages on demand via page faults; kernel kmalloc() goes through SLUB, vmalloc() through virtual mapping; both get pages from the buddy allocator.

Buddy Allocator Overview

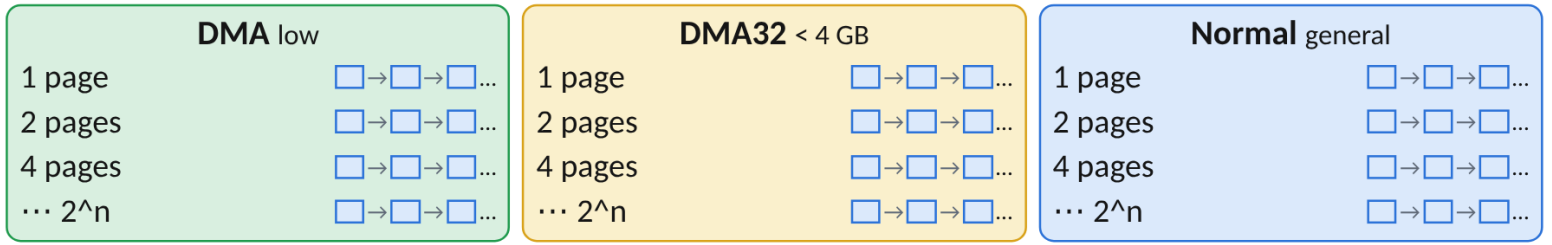
Key Points

- **Buddy Allocator**
  Linux hands out physical pages in blocks of **2^n contiguous pages**.
- **Allocation**
  A request **splits** a larger block until it matches the requested size.
- **Free**
  A freed block **merges** with its free buddy into a larger block.
- **Free Lists**
  Each zone keeps **one free list per block size**.
- **Why It Is Fast**
  A block is found by **order**; its buddy by **address arithmetic**.

Split on Allocation, Merge on Free



Free Lists per Zone



Each zone keeps **one free list per block size**; each is split by migratetype later:
`free_area[order].free_list[type]`.

Key Takeaway

The buddy allocator manages each zone as power-of-two blocks: blocks split and free buddies merge, which keeps large blocks available and allocation fast.

What Is an Order?

Key Points

- **What Is an Order?**
  An **order** *n* names a block of **2^n contiguous pages**.
- **Order and Pages**
  Pages = 2^order, so **order = log2(pages)**: order 0 is one page, order 3 is eight.
- **Choosing an Order**
  The caller selects an order large enough for the pages it needs.
- **Free List per Order**
  Each zone keeps **one free list for every order**.
- **Unused Pages**
  A larger order can leave **unused pages**, such as 3 of 8.

What Is an Order?

An **order** *n* names a block of **2^n contiguous pages**, so **order = $\log_2$(pages)**. Each zone keeps **one free list per order**.

Order	Pages (2^order)	Block size	Free list	Block
0	1	4 KiB	`free_area[0]`	
1	2	8 KiB	`free_area[1]`	
2	4	16 KiB	`free_area[2]`	
3	8	32 KiB	`free_area[3]`	
4	16	64 KiB	`free_area[4]`	… (16 pages)
10	1024	4 MiB	`free_area[10]`	typical maximum

From Pages to Order

The caller passes an **order**, not a byte count. For *N* pages, it chooses the smallest order that fits:

order = ceil($\log_2$ N)

= get_order(N * PAGE_SIZE)

- 1 page → order **0**
- 5 pages → 8 → order **3**
- 16 pages → order **4**

Why Order Matters

- The requested order **selects the free list** to search.
- An order larger than needed **wastes memory**: 5 pages → order 3 → 8 pages (3 unused).

Key Takeaway

An order n is a block of 2^n contiguous pages; the caller selects an order large enough for the pages it needs, and each order has its own free list in every zone.

Buddy Allocation API

Key Points

- **Core Call**
  alloc_pages(gfp_mask, order) returns a block of **2^order pages**.
- **Helper Calls**
  alloc_page() and **get_zeroed_page()** are shortcuts; free with **__free_pages()**.
- **Allocation Flags**
  gfp_mask sets **which zone** to use and **whether it may sleep**.
- **Two Presets**
  GFP_KERNEL may sleep; **GFP_ATOMIC** must not, so it **may fail**.

Core Call: alloc_pages()

```
struct page *alloc_pages(gfp_t gfp_mask,
  unsigned int order);
```

- Returns the **first page** of a 2^order block, or **NULL** on failure
- **gfp_mask** picks the **zone**; **order** picks the **free list** in it

Helpers and Free

Call	Gives
`alloc_page(gfp_mask)`	1 page
`__get_free_pages(gfp_mask, order)`	2^order pages
`get_zeroed_page(gfp_mask)`	1 zeroed page
`__free_pages(page, order)`	free a block

Helpers **build on alloc_pages()**; the two **__get/get** calls return an **address**. Free with the **same order**.

gfp_mask: Allocation Flags

The caller tells the allocator **how** to allocate:

- **Zone**: which zone to use (**GFP_DMA**, GFP_DMA32, or normal)
- **Sleeping**: may it sleep and reclaim memory?

Two common presets:

	GFP_KERNEL	GFP_ATOMIC
May sleep?	Yes	No
Used in	process context	interrupt, softirq
Success	higher (can wait)	**may fail**

Wrong flag: a **sleeping flag in atomic context** can crash the kernel.

Example: normal vs. DMA zone

```
// 4 pages, normal zone
p = alloc_pages(GFP_KERNEL, 2);
// 8 pages, DMA zone
a = __get_free_pages(GFP_KERNEL|GFP_DMA,3);
```

Key Takeaway

alloc_pages(gfp_mask, order) is the core buddy API; gfp_mask sets the zone and whether it may sleep: GFP_KERNEL may sleep, GFP_ATOMIC may not.

Buddy Allocator: Splitting and Coalescing

Key Points

- **Splitting**
  With no exact block, a **larger block is split** into two buddies.
- **Leftover Buddy**
  The unused buddy goes to the **lower-order free list**.
- **Coalescing**
  On free, a block **merges with its free buddy** and moves up an order.
- **Buddy Address**
  The buddy is found by **flipping one address bit** (XOR).
- **Merge Conditions**
  Blocks merge only if they are **adjacent, free, and in the same zone**.

Split on Allocation: 1 Page from Order 2

1. Find a block free order-2 block

order 2 (4 pages)

2. Split into buddies two order-1 blocks

order 1 (2 pages)	order 1 (2 pages)

3. Split again, allocate buddy stays free

Allocated	Free	order 1 (free)

Coalesce on Free: 1 Page Back to Order 2

1. Free A B, C+D already free

A freed	B free	C+D (order 1)

2. Merge with buddy B A+B becomes order 1

A+B (order 1)	C+D (order 1)

3. Merge again C+D is free → order 2

A+B+C+D (order 2)

Algorithm

- **Split**: find a free block of order ≥ requested; split until it fits; the unused buddies go to lower-order free lists.
- **Coalesce**: while the buddy is free, merge and go up one order; then insert into that order's free list.

Buddy Address

```
buddy = P XOR (1 << (N + PAGE_SHIFT))
```

The buddy of an order-N block is the **adjacent block of the same order**. Order 0 at 0x0000 → buddy 0x1000.

Merge Only If

- They are **buddies** (adjacent, same order).
- Both are **free**.
- Both are in the **same zone**.

Key Takeaway

Allocation splits a larger block into buddies, and free merges free buddies back up, reducing fragmentation while keeping large blocks available.

Problem of Buddy Allocator: External Fragmentation

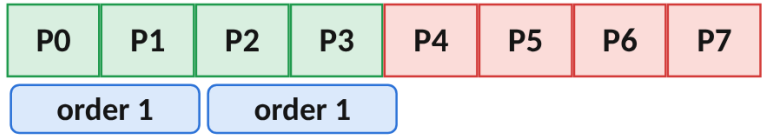
Key Points

- **Buddy Merging**
  Free **buddies merge** to keep large blocks available.
- **Not Buddies**
  Adjacent free blocks that are **not buddies** never merge.
- **External Fragmentation**
  Enough contiguous free pages, but a **high-order request fails**.
- **Impact**
  Large contiguous allocations, such as **huge pages** and DMA buffers, suffer.
- **What Next**
  The next slides show how the kernel reduces it.

Example: Order-2 Request (4 Pages)

in use free (8 pages, P0 to P7)

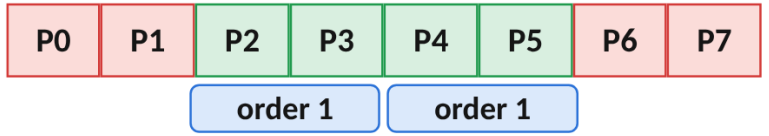
1. Buddies merge



[P0,P1] and [P2,P3] are **buddies**: both free, so they **merge into order 2**.

Order-2 request succeeds

2. Adjacent, but not buddies



[P2,P3] and [P4,P5] are **adjacent but not buddies**: the buddy of [P2,P3] is [P0,P1], and the buddy of [P4,P5] is [P6,P7]. Both are in use, so **nothing merges**.

Order-2 request fails: 4 contiguous free pages, but no order-2 block

Consequences

1. High-order allocations fail
huge pages, large DMA buffers

2. Contiguity is required
some workloads need physically contiguous memory

Observing It

`cat /proc/buddyinfo` (Normal zone, illustrative): number of **free blocks** per order

Order →	0	1	2	3	4	5	Pages
At boot	4	3	5	6	8	10	**526**
Later	206	100	30	**0**	**0**	**0**	**526**

Both show the **same 526 free pages**, but later **no block of order 3 or higher** is left. Compare the **distribution**, not the counts.

Key Takeaway

Buddy merging reduces external fragmentation but cannot remove it: adjacent free blocks that are not buddies never merge, so high-order requests can still fail.

Solutions: Migratetypes

Key Points

- **Pageblock Tags**
  Memory is divided into **pageblocks**, each with one tag. At boot, all are **movable**.
- **Free Lists per Type**
  Each order keeps **a free list per migratetype**.
- **Order and Type**
  The block comes from **that type's list**, split if larger.
- **Fallback**
  With none at any order, it **takes the largest block** of another type; the tag may change.
- **Kernel Differences**
  Rules **vary with the kernel version**.

Pageblocks and Free Lists

Each **pageblock** has one tag.

PB0	PB1	PB2	PB3	PB4	PB5
U	M	M	R	M	U

A **free block** joins the list of **its pageblock's tag**.

`free_area[2]`
`.free_list[type]`
`(blocks in PBn)`

Unmovable	PB0 PB5
Movable	PB1 PB2 PB4
Reclaimable	PB3

Allocation by Order and Type

- The caller sets **order** and **type** (**gfp_mask**):

`order`	→	`free_area[order]`
`GFP_KERNEL`	→	Unmovable
`__GFP_MOVABLE`	→	Movable
`__GFP_RECLAIMABLE`	→	Reclaimable

- Taken from **that type's list** at this order or larger.

Fallback: No U Block at Any Order

Before: all pageblocks are movable, so **no U block exists** at any order



A. Mostly free PB1: steal it, re-tag U



B. Mostly used PB0: take one, tag stays



Key Takeaway

Each request is served from pageblocks of its own type; when none is free, fallback takes a block from another type, so types can mix.

Solutions: Compaction

Key Points

- **Before Compaction**
  Free pages are **split among partly used pageblocks**.
- **Page Migration**
  Compaction **migrates movable pages** to the other end.
- **Free Blocks**
  A **movable-only** pageblock becomes one contiguous free block.
- **Role of Migratetypes**
  Unmovable pages are in **fewer pageblocks**, so more can be emptied.

Before Compaction

Free pages are **split among** partly used pageblocks.



No pageblock-sized free block exists.

Blue frame = pageblock (really 2 MiB)

After Compaction

Movable pages are **migrated** to the other end.



PB1 held only movable pages, so it becomes **one free block**.

What Compaction Achieves

- **Movable-only** pageblocks can be emptied (PB1).
- A pageblock with an **unmovable** page cannot be emptied (PB2).
- A **high-order request** can now use the free block.
- **Cost**: page migration uses **CPU time and memory bandwidth**.

Why Migratetypes Help

- Without migratetypes, unmovable pages use **any free block**, in **many pageblocks**.
- With migratetypes, they use **U pageblocks first**, in **fewer pageblocks**.
- Fallback can still **put an unmovable page in a movable pageblock**: **reduced, not removed**.

Key Takeaway

Compaction migrates movable pages and frees whole pageblocks; migratetypes limit the pageblocks that hold unmovable pages.

Contiguous Memory for Devices: CMA

Key Points

- **Devices Need Contiguity**
  Cameras, codecs, and displays need **large contiguous buffers**.
- **CMA Region**
  A region is **reserved at boot**; only movable pages may use it while idle.
- **Device Buffers**
  cma_alloc() migrates those pages out and allocates a contiguous buffer.
- **Limits**
  Pinned pages block migration, so an allocation can be slow or fail.

1 Why Devices Need CMA

- Devices need **physically contiguous buffers** of many MiB.
- The buddy allocator has a **maximum order** (kernel-configured), and fragmentation can make large blocks unavailable even when free memory is enough.
- A permanent reservation **wastes memory** while idle.

3 Limits of CMA

- **Pinned pages** cannot be migrated: **cma_alloc()** tries other ranges, then returns **NULL**.
- **Migration** copies pages, which adds delay.
- Free CMA memory serves **only movable** pages.

2 How CMA Works

Idle: movable pages use the CMA region



cma_alloc(): those pages are migrated out



Device buffer: one contiguous range



- Reserved at boot, it **overlaps buddy-managed memory**; the buddy gives it **only to movable pages**.
- A driver calls `dma_alloc_coherent()` for a DMA buffer; it calls **cma_alloc()**.
- The size is a **configuration value** (kernel default about 16 MiB); see `CmaTotal` in `/proc/meminfo`.

Key Takeaway

CMA reserves a region that movable pages use while idle; cma_alloc() migrates them out to allocate large contiguous buffers.

Buddy Allocator: Allocation Path (Simplified)

Key Points

- **Three Stages**
  The **fast path** comes first, then the **slow path**, then failure handling.
- **Zone Selection**
  gfp_mask decides the zone order.
- **Watermark Check**
  Free pages must be **above the low watermark**.
- **Order Lookup**
  The block comes from **free_area[order]**, split if larger.
- **Later Weeks**
  Slow path: Week 9. OOM and recovery: Week 10.

Allocation Path: Three Stages

This week **Fast path** above the watermark → Week 9 **Slow path** reclaim, compaction → Week 10 **Failure** OOM killer, low-memory killer

Fast Path Steps (Simplified)

1. **Select zone** — **gfp_mask** gives the zone list; try the preferred zone first.
2. **Watermark** — Free pages must be **above the low watermark**; if not, next zone.
3. **Order** — Take a block from `free_area[order]`; **split** a larger block if needed.
4. **Return** — The caller gets the page.

When the Fast Path Fails

- No zone has free pages **above the low watermark**.
- No free block of the **requested order** (external fragmentation).
- The request moves to the **slow path** (Week 9).

Simplified path. Linux also uses per-CPU page caches to accelerate small-page allocations.

Key Takeaway

The fast path selects a zone, checks the low watermark, takes a page by order, and returns it; failures go to the slow path (Week 9).

Observing the Buddy Allocator

Key Points

- **Free Blocks per Order**
  /proc/buddyinfo shows free blocks per zone and order.
- **Watermarks**
  /proc/zoneinfo shows min, low, and high per zone.
- **Free Blocks per Type**
  /proc/pagetypeinfo adds the migratetype.
- **Pageblocks per Type**
  It also counts **pageblocks** by type.
- **CMA**
  /proc/meminfo shows CmaTotal and CmaFree when CMA is enabled.

1. /proc/buddyinfo and /proc/zoneinfo

```
$ cat /proc/buddyinfo
order      0   1   2   3   4   5   6   7   8   9  10
DMA32      0   2   3   1   2   4   2   3   4   2 753
Normal   982 357 230 120 325 289 182 108 133  90 108
$ cat /proc/zoneinfo
zone Normal
  pages free     234116
  min      6803
  low      8503
  high    10203
  cma      0
```

How to Read

- **buddyinfo**: column **n** = free blocks of **order n** (2^n pages); order 10 = 4 MiB.
- **Fragmentation** needs a baseline: compare the **share of free pages** (count × 2^{order}) in high-order blocks **over time**.
- **zoneinfo**: **min, low, high** watermarks (pages); the fast path needs free pages above **low**.

2. /proc/pagetypeinfo: Free Blocks by Migratetype

```
$ cat /proc/pagetypeinfo
order                    0   1   2   3   4   5   6   7   8   9  10
Normal Unmovable       153 168 166  83  40  14  11  13   6   0   0
Normal Movable         828 189  63  37 285 274 170  93 126  90 108
Normal Reclaimable       1   0   1   0   0   1   1   2   1   0   0
Blocks     Unmovable  Movable  Reclaimable
Normal            45      971            8
```

- **Upper table**: free blocks per order and type (**free_list[type]**). **Lower table**: pageblocks per type.

Example environment (some labels shortened): Linux 6.18, x86-64 VM, **CMA off**, pageblock order 9 (2 MiB). **Values differ** by kernel version, configuration, and workload.

Key Takeaway

Use /proc/buddyinfo, /proc/zoneinfo, and /proc/pagetypeinfo to see free blocks by order and type, and the watermarks the fast path checks.

Why is Buddy Allocator Not Enough?

Key Points

- **Page granularity**
  The buddy allocator works in **pages** (e.g., 4 KiB).
- **Small objects**
  Many kernel structures are **tens to hundreds of bytes**.
- **Waste**
  One page per object means **severe memory waste**.
- **Overhead**
  Frequent page allocation also costs **time**.
- **Next Step**
  A dedicated allocator, **SLUB**, stores many objects in one page.

Small Object via Buddy

Allocate a **128-byte** kernel object from the buddy allocator:



- One **4 KiB page** holds just 128 B.
- **More than 96%** of it is unused: severe **internal fragmentation**.

Reality: Many Small Objects

Structure	Size	Usage
`task_struct`	2–6 KiB	one per thread
`inode`	~ 600 B	one per file
`dentry`	~ 192 B	one per dir entry
`file`	~ 256 B	one per open file
`sk_buff`	~ 256 B	packet
`timer_list`	~ 64 B	kernel timer

- Allocated and freed **very often**.
- Sizes are **approximate**; see `/proc/slabinfo`.

Problems with Buddy Alone

1. Memory waste
a small object wastes most of a page

2. High overhead
frequent page allocation adds locking and cache cost

3. Not scalable
millions of objects to manage

Key Takeaway

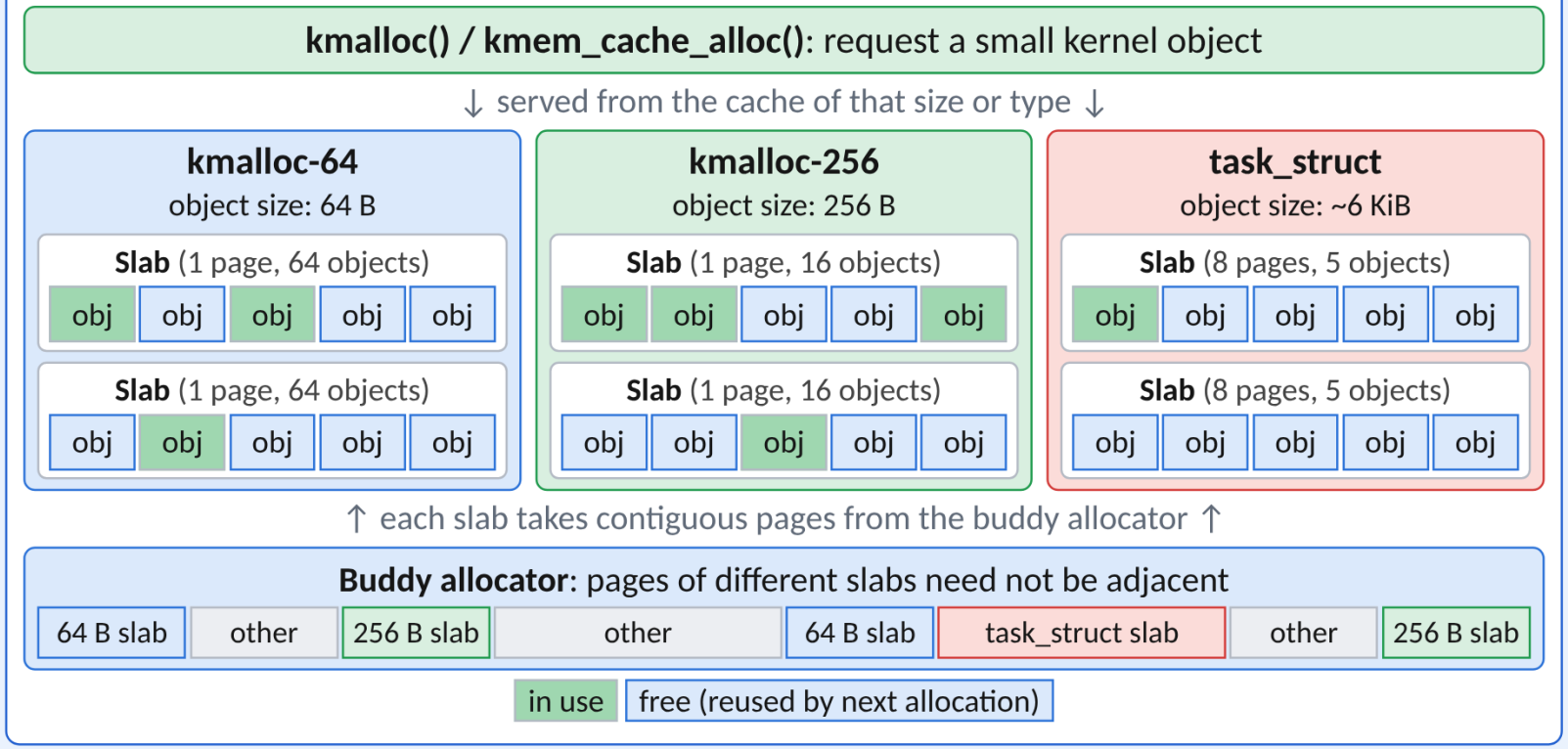
The buddy allocator works at page granularity, which is inefficient for many small objects; a dedicated allocator such as SLUB is needed.

SLUB Allocator Overview

Key Points

- **Slab**
  A **slab** is one or more contiguous pages split into same-size objects.
- **Cache**
  A **cache** is the set of slabs for one object size or type.
- **Buddy Pages**
  Each slab gets contiguous pages from the **buddy allocator**.
- **Fast Reuse**
  Freed objects stay in their slab and are **reused**.
- **SLUB**
  SLUB is the current slab implementation in Linux.

Caches, Slabs and Objects



Slab and SLUB

Earlier Linux versions used the **SLAB** and **SLOB** allocators. **SLUB** was designed with **simpler metadata** and better **multicore scalability**, so recent Linux uses SLUB.

Key Takeaway

A slab stores many same-size objects in contiguous pages, so small kernel objects are allocated quickly and reused.

kmalloc() and kfree(): Generic Caches

Key Points

- **kmalloc()**
  Allocates a **kernel object** of the requested size.
- **Size Classes**
  It uses **generic caches** such as kmalloc-64 and kmalloc-256.
- **Rounding Up**
  The size is rounded up to the **next cache size**.
- **Large Requests**
  Above the largest cache (**8 KiB**, typical), requests go to the **buddy allocator**.
- **kfree()**
  Memory from **kmalloc()** is released with **kfree()**.

kmalloc(size, gfp): Size Selects the Cache

`kmalloc(40, gfp)`	→	**kmalloc-64**	24 B unused
`kmalloc(100, gfp)`	→	**kmalloc-128**	28 B unused
`kmalloc(200, gfp)`	→	**kmalloc-256**	56 B unused
`kmalloc(5000, gfp)`	→	**kmalloc-8k**	3192 B unused
`kmalloc(1 MiB, gfp)`	→	**Buddy alloc.**	above 8 KiB

Rounded up to the next cache size. Largest cache: kmalloc-8k.

Real Values: Slabs of kmalloc Caches

Cache	Object size	Objects / slab	Pages / slab	Slabs
`kmalloc-64`	64 B	**64**	1	25
`kmalloc-256`	256 B	**16**	1	38
`kmalloc-1k`	1 KiB	**16**	4	27
`kmalloc-8k`	8 KiB	**4**	8	8

One example; values vary by kernel version. Check: `/sys/kernel/slab/<cache>/` or `/proc/slabinfo` (root).

kmalloc Limit and Uses

- **Maximum size** depends on the kernel configuration and architecture; physically contiguous.
- Larger or non-contiguous buffers: `vmalloc()` (virtually contiguous).
- **Typical uses:** driver data, variable-size buffers, small temporary data.
- task_struct, inode, dentry use **dedicated caches** (`kmem_cache`).

Key Takeaway

kmalloc() rounds the size up to a generic cache; memory from kmalloc() is released with kfree().

kmem_cache: Dedicated Caches

Key Points

- **kmem_cache**
  A **kmem_cache** is one cache for one object type.
- **Create**
  kmem_cache_create() sets name, size, alignment and constructor.
- **Allocate and Free**
  kmem_cache_alloc() and **kmem_cache_free()** take the cache as argument.
- **Constructor**
  It initializes each object **once**, when the slab is set up.
- **Why Dedicated**
  Exact size, alignment and statistics per type.

API: One Cache per Object Type

```
/* lifecycle */
cache = kmem_cache_create(
  "my_obj", size, align,
  flags, ctor);
obj = kmem_cache_alloc(
  cache, GFP_KERNEL);
kmem_cache_free(cache,
obj);
kmem_cache_destroy(cache);
```

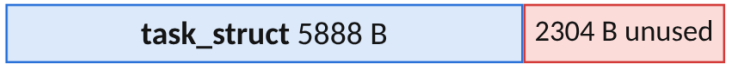
Example: Dedicated Caches

Cache	Object size	Objects / slab	Pages / slab	Constructor
`task_struct`	5888 B	**5**	8	-
`mm_struct`	1600 B	**20**	8	-
`ext4_inode_cache`	1120 B	**14**	4	init_once
`dentry`	192 B	**21**	1	-
`anon_vma`	104 B	**39**	1	anon_vma_ctor

One example; values vary by kernel version. Check: `/sys/kernel/slab/<cache>/` or `/proc/slabinfo` (root).

Why Use a Dedicated Cache?

`kmalloc(5888)`: object is 8192 B (kmalloc-8k)



`kmem_cache` for task_struct: object is 5888 B



- **Exact size**: no space is wasted by rounding up to a kmalloc size.
- **Constructor**: sets up each object once, when the slab is created, so each allocation is faster.
- **Statistics**: `/proc/slabinfo` shows the memory used by each type.

Key Takeaway

kmem_cache is a dedicated cache for one object type, such as task_struct or inode.

Choosing a Kernel Allocation API

Key Points

- **Many APIs**
  Linux provides several allocation APIs for **different needs**.
- **kmalloc()**
  Small, **physically contiguous** memory from SLUB; kzalloc() zeroes it.
- **alloc_pages()**
  One or more **contiguous pages** from the buddy allocator.
- **vmalloc()**
  Virtually contiguous memory (physical pages may be scattered).
- **Choose**
  By **size, contiguity, and context**.

	`kmalloc()`	`kzalloc()`	`alloc_pages()`	`vmalloc()`
Typical size	bytes to KiB (size limit)	bytes to KiB (size limit)	page multiples (max order)	large (no such limit)
Contiguity	physically contiguous	physically contiguous	physically contiguous	virtually contiguous only
Allocator	SLUB (large: buddy)	SLUB (large: buddy)	buddy allocator	vmalloc area
Returns	void *	void *	struct page *	void *
Zeroed?	No	Yes	No (use __GFP_ZERO)	No (use vzalloc())
Typical use	small objects, buffers	small objects, zeroed memory	page-level buffers, DMA	large buffers

Key Takeaway

Use kmalloc()/kzalloc() for small objects, alloc_pages() for physically contiguous pages, and vmalloc() for large virtually contiguous regions.

Observing Slab Memory: slabinfo and meminfo

Key Points

- **Per-Cache Usage**
  /proc/slabinfo lists objects and slabs for each cache.
- **Reading Columns**
  active is in use; **num** is allocated.
- **Cache Memory**
  Memory is **slabs × pages per slab × page size**.
- **Sorted View**
  slabtop sorts the caches, for example by size.
- **Total Slab Memory**
  /proc/meminfo shows **Slab**, split into SReclaimable and SUnreclaim.

1. /proc/slabinfo: Per-Cache Usage

```
$ sudo cat /proc/slabinfo
# name            active num   objsize objs pages  slabs
task_struct          111   140   5888   5    8     28
dentry              3677  3927    192  21    1    187
kmalloc-64          1431  1536     64  64    1     24
```

- **active**: objects in use. **num**: objects allocated.
- **objs**, **pages**: objects and pages per slab; **slabs**: slabs in the cache.
- Cache memory = slabs × pages × 4 KiB (task_struct: 28 × 8 × 4 KiB = 896 KiB).

2. slabtop: Caches Sorted by Size

```
$ sudo slabtop -o -s c
OBJS  ACTIVE  USE  SLABS  CACHE SIZE  NAME
  648     602  92%     81       2592K  kmalloc-4k
 1624    1624 100%    116       1856K  ext4_inode_cache
  140     111  79%     28        896K  task_struct
```

- **slabtop** reads /proc/slabinfo and sorts the caches (**-s c**: by cache size).
- **USE** = ACTIVE / OBJS; **CACHE SIZE** = memory used by the cache.

3. /proc/meminfo: Total Slab Memory

```
$ cat /proc/meminfo
...
Slab:           25620 kB
SReclaimable:    6428 kB
SUnreclaim:     19192 kB
```

- **Slab** = SReclaimable + SUnreclaim.
- **SReclaimable**: caches such as dentry and inode caches; can be freed under memory pressure (Week 9).
- Different counters: values need not match slabinfo exactly.

Example environment: Linux 6.18, x86-64 VM. **Values differ** by kernel version, configuration, and workload.

Key Takeaway

Use /proc/slabinfo or slabtop for each cache, and /proc/meminfo for the total slab memory.

Checkpoint: Which Allocator?

Checkpoint

- **Match the need**
  Pick the API by **size, contiguity, and context**.
- **Small objects**
  kmalloc()/kzalloc() for small, contiguous objects (SLUB).
- **Pages**
  alloc_pages() for page-level, physically contiguous memory.
- **Large**
  vmalloc() for large, virtually contiguous memory.
- **Context**
  Use **GFP_ATOMIC** where sleeping is not allowed; prefer the DMA API for devices.

For each scenario, choose the most appropriate **allocation API and GFP flags**.

#	Scenario	Constraints	Recommended API (flags) and why
1	Small buffer for network packet data	~1 KB; process context	`kmalloc(size, GFP_KERNEL)` Small, contiguous, normal context.
2	Allocate and zero a small structure	128 B; process context	`kzalloc(size, GFP_KERNEL)` Small object that must be zeroed.
3	8 contiguous pages for a legacy DMA buffer	32 KB; in the DMA zone (< 16 MB)	`alloc_pages(GFP_DMA, 3)` 8 contiguous pages = order 3, DMA zone.
4	Large buffer for temporary kernel data	10 MB; virtual contiguity is enough	`vmalloc(size)` Large; physical contiguity not needed.
5	Memory in an interrupt handler	256 B; cannot sleep	`kmalloc(size, GFP_ATOMIC)` Atomic context: must not sleep.
6	One page, zero-initialized	4 KB; process context	`get_zeroed_page(GFP_KERNEL)` One page, zeroed.

Key Takeaway

Choose the allocation API and GFP flags by size, contiguity, and context. There is no single right answer: understand the trade-offs.

Reducing Memory Use: Targets and Trade-offs

Key Points

- **Where Memory Goes**
  RAM holds **process memory**, **file cache**, and **kernel memory**.
- **Reduce with a Cost**
  Every reduction has a **trade-off**, such as more CPU work.
- **No Swap**
  Without swap, **heap and stack** stay in RAM.
- **Fragmentation and CMA**
  A smaller **CMA** can make device buffers fail.
- **Measure on Target**
  Check **meminfo** on the real device.

What to Reduce	How	Trade-off	Caution and Tools
Process memory (heap, stack)	Smaller buffers; load data on demand; share libraries	More CPU or I/O work later	No swap: the peak of every service must fit in RAM. Check PSS in `/proc/<pid>/smaps`.
File cache (page cache)	Reduce its size	More reads from storage; slower file access	Clean file cache can be freed without swap. Too small a cache causes repeated reads.
Kernel memory	Remove unused drivers and features; use exact-size caches	Fewer features	Check `Slab`, `PageTables`, `KernelStack` in /proc/meminfo.
CMA and reserved memory	Reserve a smaller region	Device buffer allocation may fail or become slower	Size depends on the product. Check `CmaTotal` in /proc/meminfo.
Free-memory headroom	Lower `vm.min_free_kbytes` (watermarks)	Smaller reserve: atomic allocations may fail; reclaim may stall under load	Change only after measuring on the target.

Key Takeaway

Reduce memory where the trade-off is acceptable, keep headroom for device buffers, and measure on the target device before and after.

Week 8 Summary

1. Address Translation

Virtual address
from the process (stack, mmap, heap, ...)

↓

TLB lookup and page table walk
hit: physical address; miss: check PTE

↓

Not present or invalid
→ page fault handling

2. Physical Page Allocation

Allocation request
alloc_pages(gfp_mask, order)

↓

Buddy allocator
fast path above the low watermark;
free_area[order], split and merge

↓

Contiguity
migratetypes, compaction, CMA

3. Kernel Object Allocation

Kernel subsystems
VFS, networking, timers, ...

↓

kmalloc() / kmem_cache_alloc()
generic size classes / one cache per object type

↓

SLUB caches
cache, slabs, objects, freelist; slab pages come from the buddy allocator

Key Takeaways

1. Address translation
multi-level page tables and the TLB; page size trades TLB reach for granularity

2. Page faults
the kernel allocates or maps a page, updates the PTE, and retries

3. Physical memory allocator
buddy allocator for pages (zones, orders, migratetypes, watermarks); SLUB for kernel objects (kmalloc, kmem_cache, slabs)

4. Memory budget
reduce memory only where the trade-off is acceptable; measure on the target

Next: Week 9, Memory Pressure and Page Reclaim

What happens when Linux cannot find enough free physical pages?

- Memory pressure and its causes
- Page reclaim (kswapd and direct reclaim)
- Reclaimable vs. unreclaimable memory
- LRU and eviction policies
- Observing memory pressure in practice

End of Week 8

Sungyong Park, Ph.D. | **parksy@sogang.ac.kr**



